

React Testing — Master Notes

Installation • Config • Imports • Real-World Scenarios

ReactDOM • RTL • Jest • Vitest • MSW • userEvent • renderHook • Cypress

Chapter 1 — Project Setup: Jest + React Testing Library

Start here for CRA projects, Webpack-based setups, or any non-Vite project.

1.1 Installation

Option A — Create React App (Jest pre-installed)

```
• • •
# CRA comes with Jest + RTL out of the box
npx create-react-app my-app
cd my-app

# RTL packages already installed:
# @testing-library/react
# @testing-library/jest-dom
# @testing-library/user-event
```

Option B — Manual Install (Existing Project)

```
• • •
# Core testing stack
npm install --save-dev jest jest-environment-jsdom
npm install --save-dev @testing-library/react
npm install --save-dev @testing-library/jest-dom
npm install --save-dev @testing-library/user-event

# For JSX/TypeScript transform
npm install --save-dev babel-jest @babel/core @babel/preset-env @babel/preset-react
npm install --save-dev @babel/preset-typescript # if using TS

# CSS Modules mock
npm install --save-dev identity-obj-proxy

# HTTP mocking
npm install --save-dev msw

# Test data factories
npm install --save-dev @jackfranklin/test-data-bot @faker-js/faker
```

1.2 babel.config.js

```
• • •
// babel.config.js — needed for Jest to understand JSX + modern JS
module.exports = {
  presets: [
    ['@babel/preset-env', { targets: { node: 'current' } }],
    ['@babel/preset-react', { runtime: 'automatic' }],
    '@babel/preset-typescript', // only if using TypeScript
  ],
}
```

1.3 jest.config.js — Complete Configuration

```
• • •
// jest.config.js
module.exports = {
```

```
// Use jsdom to simulate browser environment in Node
testEnvironment: 'jsdom',

// Run setup files AFTER jest + testing framework loads
setupFilesAfterFramework: [
  '@testing-library/jest-dom', // adds .toBeInTheDocument() etc.
  '<rootDir>/src/setupTests.js', // your custom global setup
],

// Tell Jest how to transform files
transform: {
  '^.+\\. (js|jsx|ts|tsx)$': 'babel-jest',
},

// Mock non-JS imports that Jest can't handle
moduleNameMapper: {
  // CSS Modules → identity-obj-proxy returns className as-is
  '\\.(css|scss|sass|less)$': 'identity-obj-proxy',
  // Static files → simple mock
  '\\.(png|jpg|jpeg|gif|svg|webp|ico|woff|woff2|ttf|eot)$':
    '<rootDir>/__mocks__/fileMock.js',
  // Path aliases (if you have @ → src)
  '^@/(.*)$': '<rootDir>/src/$1',
},

// Which files to collect coverage from
collectCoverageFrom: [
  'src/**/*. {js,jsx,ts,tsx}',
  '!src/**/*.d.ts',
  '!src/index. {js,tsx}',
  '!src/reportWebVitals. {js,ts}',
  '!src/**/*.stories. {js,jsx,ts,tsx}',
],

// Minimum coverage thresholds
coverageThreshold: {
  global: { branches: 70, functions: 70, lines: 70, statements: 70 },
},

// Where Jest looks for tests
testMatch: [
  '**/ __tests__/**/*. {js,jsx,ts,tsx}',
  '**/*. {spec,test}. {js,jsx,ts,tsx}',
],

// Files to ignore
testPathIgnorePatterns: ['/node_modules/', '/dist/', '/build/'],

// Automatically clear mock calls between every test
clearMocks: true,
}
```

1.4 __mocks__/fileMock.js

```
// __mocks__/fileMock.js – at root level
module.exports = 'test-file-stub'
```

1.5 src/setupTests.js

```
// src/setupTests.js – runs before every test file
```

```
// 1. Import jest-dom matchers
import '@testing-library/jest-dom'

// 2. Global console error suppression for known React warnings (optional)
const originalError = console.error
beforeAll(() => {
  console.error = (...args) => {
    if (/Warning.*not wrapped in act/.test(args[0])) return
    originalError(...args)
  }
})
afterAll(() => { console.error = originalError })

// 3. MSW global server (if you want it for all tests)
// import { server } from './test/mocks/server'
// beforeAll(() => server.listen({ onUnhandledRequest: 'warn' }))
// afterEach(() => server.resetHandlers())
// afterAll(() => server.close())
```

1.6 package.json Scripts

```
{
  "scripts": {
    "test": "jest --watchAll=false",
    "test:watch": "jest --watch",
    "test:coverage": "jest --coverage",
    "test:ci": "jest --ci --coverage --watchAll=false",
    "test:verbose": "jest --verbose"
  }
}
```

Chapter 2 — Project Setup: Vitest + React Testing Library

For Vite-based projects (create-vite, Vite + React, Vite + React + TypeScript).

2.1 Installation

```
# Create project
npm create vite@latest my-app -- --template react-ts
cd my-app
npm install

# Testing deps
npm install --save-dev vitest
npm install --save-dev @vitest/coverage-v8           # coverage
npm install --save-dev @vitest/ui                   # visual test browser UI
npm install --save-dev jsdom                       # DOM simulation
npm install --save-dev @testing-library/react
npm install --save-dev @testing-library/jest-dom
npm install --save-dev @testing-library/user-event
npm install --save-dev msw
npm install --save-dev @jackfranklin/test-data-bot @faker-js/faker
```

2.2 vite.config.ts — Complete

```
// vite.config.ts
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import path from 'path'

export default defineConfig({
  plugins: [react()],

  // Path aliases (optional)
  resolve: {
    alias: { '@': path.resolve(__dirname, './src') },
  },

  // — Vitest config lives here —————
  test: {
    // jsdom = simulated browser; 'happy-dom' is a faster alternative
    environment: 'jsdom',

    // Makes describe/test/expect etc. available without importing
    globals: true,

    // Runs before EVERY test file
    setupFiles: ['./src/test/setup.ts'],

    // Which files are test files
    include: ['**/*.test.ts', '**/*.spec.ts', '**/*.ts', '**/*.tsx', '**/*.js', '**/*.jsx'],
    exclude: ['node_modules', 'dist', 'cypress', 'e2e'],

    // Coverage config
    coverage: {
      provider: 'v8',
      reporter: ['text', 'json', 'html', 'lcov'],
      include: ['src/**'],
    },
  },
})
```

```

    exclude: [
      'src/**/*.d.ts',
      'src/main.tsx',
      'src/**/*.stories.*',
      'src/test/**',
    ],
    thresholds: {
      branches: 70, functions: 70, lines: 70, statements: 70,
    },
  },
  // Show test names in output
  reporters: ['verbose'],
},
})

```

2.3 src/test/setup.ts

```

// src/test/setup.ts - runs before every test file
import '@testing-library/jest-dom'

// If you want MSW globally for all tests:
// import { server } from './mocks/server'
// beforeAll(() => server.listen({ onUnhandledRequest: 'error' }))
// afterEach(() => server.resetHandlers())
// afterAll(() => server.close())

```

2.4 tsconfig.json — TypeScript support for globals

```

{
  "compilerOptions": {
    "types": ["vitest/globals", "@testing-library/jest-dom"],
    "paths": { "@/*": [".src/*"] }
  }
}

```

2.5 package.json Scripts

```

{
  "scripts": {
    "test": "vitest",
    "test:run": "vitest run",
    "test:ui": "vitest --ui",
    "test:coverage": "vitest run --coverage",
    "test:ci": "vitest run --coverage --reporter=verbose"
  }
}

```

2.6 MSW Setup (shared by Jest & Vitest)

```

# Generate MSW service worker for browser dev mode
npx msw init public/ --save

// src/test/mocks/handlers.ts
import { http, HttpResponse } from 'msw'

export const handlers = [

```

```
http.post('https://api.example.com/login', async ({ request }) => {
  const { username, password } = await request.json()
  if (!password) return HttpResponse.json({ message: 'password required' }, {
status: 400 })
  if (!username) return HttpResponse.json({ message: 'username required' }, {
status: 400 })
  return HttpResponse.json({ username })
}),

http.get('https://api.example.com/users', () => {
  return HttpResponse.json([
    { id: 1, name: 'John' },
    { id: 2, name: 'Jane' }
  ]),

http.get('https://api.example.com/users/:id', ({ params }) => {
  const { id } = params
  return HttpResponse.json({ id: Number(id), name: 'John' })
}),
]
```

```
● ● ●
// src/test/mocks/server.ts
import { setupServer } from 'msw/node'
import { handlers } from '../handlers'

// This server runs in Node (Jest / Vitest)
export const server = setupServer(...handlers)
```

Chapter 3 — Complete Import Reference

Every import you will ever need, organised by library.

3.1 @testing-library/react

```
import {
  // Core rendering
  render, // mount component, returns queries + container
  renderHook, // mount hook without component UI
  act, // flush state updates manually

  // Screen (global query object - preferred)
  screen,

  // Queries (also available on render() return value)
  // getBy* - throws if not found
  // queryBy* - returns null if not found
  // findBy* - async, returns Promise

  // Individual query functions (rare - use screen instead)
  getByRole, getByLabelText, getByText, getByTestId,
  queryByRole, findByRole,

  // Async utilities
  waitFor, // retry assertion until passes
  waitForElementToBeRemoved, // wait for element to disappear

  // Events (basic)
  fireEvent,

  // Cleanup (automatic - rarely needed manually)
  cleanup,
} from '@testing-library/react'
```

3.2 @testing-library/user-event

```
import userEvent from '@testing-library/user-event'

// Always create instance per test
const user = userEvent.setup()

// Available methods:
// user.click(el)
// user.dblClick(el)
// user.type(el, 'text')
// user.clear(el)
// user.keyboard('{Tab}')
// user.hover(el) / user.unhover(el)
// user.selectOptions(el, value)
// user.upload(el, file)
// user.paste('text')
```

3.3 @testing-library/jest-dom

```
// Import once in setup file - extends expect() globally
import '@testing-library/jest-dom'
```

```
// After import, these matchers are available on expect():
// .toBeInTheDocument()
// .toHaveTextContent(text)
// .toHaveStyle(css)
// .toHaveAttribute(attr, value)
// .toHaveValue(value)
// .toBeDisabled() / .toBeEnabled()
// .toBeVisible()
// .toBeChecked()
// .toHaveClass(className)
// .toHaveFocus()
// .toBeRequired()
// .toBeEmptyDOMElement()
// .toHaveDisplayValue(value)
// .toContainElement(el)
// .toHaveDescription(text)
```

3.4 msw

```
// MSW v2 - Node (Jest / Vitest tests)
import { http, HttpResponse, delay } from 'msw'
import { setupServer } from 'msw/node'

// MSW v2 - Browser (dev mode service worker)
import { setupWorker } from 'msw/browser'
```

3.5 Jest globals

```
// Available globally (no import needed in CRA / with globals:true in Vitest)
describe('group', () => {})
test('name', () => {})
it('alias for test', () => {})
expect(value)
beforeAll(() => {})
afterAll(() => {})
beforeEach(() => {})
afterEach(() => {})

// Explicit import for pure ESM or TypeScript strictness
import { describe, test, expect, jest } from '@jest/globals'

// Vitest explicit import
import { describe, test, expect, vi, beforeEach, afterEach } from 'vitest'
```

3.6 Test Data

```
// Faker
import { faker } from '@faker-js/faker'

// Test-data-bot (factory pattern)
import { build, fake, sequence, oneOf, bool, perBuild } from '@jackfranklin/test-data-bot'
```

3.7 renderHook

```
import { renderHook, act } from '@testing-library/react'
```

```
// Basic
const { result } = renderHook(() => useMyHook())
result.current // current hook return value

// With initial props
const { result, rerender } = renderHook(
  (props) => useCounter(props),
  { initialProps: { initialCount: 0, step: 1 } }
)
rerender({ step: 5 }) // update props

// With context wrapper
const wrapper = ({ children }) => <ThemeProvider>{children}</ThemeProvider>
const { result } = renderHook(() => useTheme(), { wrapper })
```

Chapter 4 — screen Queries — Every Scenario

4.1 getByRole — Most Recommended

```
// Finds by ARIA role — matches how screen readers see the page
screen.getByRole('button')
screen.getByRole('button', { name: /submit/i }) // by accessible name
screen.getByRole('button', { name: 'Submit' }) // exact string
screen.getByRole('heading', { level: 1 }) // h1 only
screen.getByRole('heading', { level: 2, name: /title/i })
screen.getByRole('textbox', { name: /username/i }) // input[type=text]
screen.getByRole('checkbox', { name: /agree/i })
screen.getByRole('radio', { name: /yes/i })
screen.getByRole('combobox', { name: /country/i }) // select
screen.getByRole('link', { name: /home/i })
screen.getByRole('img', { name: /logo/i }) // needs alt text
screen.getByRole('alert') // role="alert"
screen.getByRole('dialog') // modal
screen.getByRole('listitem') // li
screen.getByRole('tab')
screen.getByRole('tabpanel')
screen.getByRole('progressbar')
```

NOTE

The "name" option matches the accessible name: button text, aria-label, aria-labelledby, or label text for inputs.

4.2 getByLabelText — For Form Fields

```
// Matches <label> text → finds the associated input
screen.getByLabelText('Username')
screen.getByLabelText(/username/i)

// Works with all label association methods:
// 1. htmlFor / id
<label htmlFor="email">Email</label>
<input id="email" type="email" />

// 2. aria-label
<input aria-label="Search" />
screen.getByLabelText(/search/i)

// 3. aria-labelledby
<span id="lbl">Email</span>
<input aria-labelledby="lbl" />
screen.getByLabelText(/email/i)

// 4. Wrapping label
<label>Email <input type="email" /></label>
screen.getByLabelText(/email/i)
```

4.3 getByText — For Non-Interactive Elements

```
screen.getByText('Hello world') // exact match
screen.getByText(/hello/i) // regex (case-insensitive)
screen.getByText(/hello/, { exact: false }) // partial string match
screen.getByText('$42.00') // for price displays
```

```
// Narrow down element type
screen.getByText(/submit/i, { selector: 'button' })
screen.getByText(/error/i, { selector: '[role="alert"]' })
```

4.4 getByPlaceholderText

```
screen.getByPlaceholderText('Enter your email')
screen.getByPlaceholderText(/enter email/i)
```

4.5 getByDisplayValue

```
// For inputs/ selects - matches current VALUE, not label
screen.getByDisplayValue('john@example.com') // input already has a value
screen.getByDisplayValue('India')           // selected option in <select>
```

4.6 getByTestId — Last Resort

```
// In component:
<div data-testid="user-card">...</div>

// In test:
screen.getByTestId('user-card')

// When to use:
// - No role/label/text to query by
// - Complex visualizations (charts, canvas)
// - Dynamic content with no predictable text
```

❗ WARN

Avoid data-testid if possible. It couples tests to markup. It will not catch accessibility issues.

4.7 queryBy — Asserting Absence

```
// Use queryBy when you expect element NOT to exist
expect(screen.queryByText(/loading/i)).not.toBeInTheDocument()
expect(screen.queryByRole('alert')).toBeNull()

// Common pattern: assert element is gone after action
fireEvent.click(closeButton)
expect(screen.queryByRole('dialog')).not.toBeInTheDocument()
```

4.8 findBy — Async Elements

```
// findBy = getBy + waitFor - retries until found or timeout
const username = await screen.findByText('John')
const btn = await screen.findByRole('button', { name: /retry/i })

// With timeout option
const el = await screen.findByText(/loaded/i, {}, { timeout: 3000 })

// findAll for multiple async elements
const items = await screen.findAllByRole('listitem')
expect(items).toHaveLength(5)
```

4.9 getAllBy — Multiple Elements

```

// Returns array - throws if none found
const buttons = screen.getAllByRole('button')
expect(buttons).toHaveLength(3)

const items = screen.getAllByRole('listitem')
items.forEach(item => expect(item).toBeVisible())

// queryAllBy - returns [] if none found (no throw)
const errors = screen.queryAllByRole('alert')
expect(errors).toHaveLength(0) // assert no errors
```

4.10 screen.debug() — Your Best Friend

```

render(<MyComponent />)
screen.debug() // pretty-prints full DOM to console
screen.debug(element) // prints specific element only

// When a query fails, debug to see actual DOM:
try {
  screen.getByRole('button', { name: /submit/i })
} catch {
  screen.debug() // see what's actually rendered
}
```

Chapter 5 — User Interaction Scenarios

5.1 Basic Click

```
...  
// fireEvent (simple, synchronous)  
fireEvent.click(screen.getByRole('button', { name: /submit/i }))  
  
// userEvent (realistic, async — preferred)  
const user = userEvent.setup()  
await user.click(screen.getByRole('button', { name: /submit/i }))
```

5.2 Typing in Input

```
...  
const user = userEvent.setup()  
const input = screen.getByLabelText(/username/i)  
  
// type — appends to existing value  
await user.type(input, 'john_doe')  
  
// clear then type  
await user.clear(input)  
await user.type(input, 'new_name')  
  
// Triple click selects all, then type replaces  
await user.tripleClick(input)  
await user.type(input, 'replacement')  
  
// Assert  
expect(input).toHaveValue('replacement')
```

5.3 Special Keys

```
...  
const user = userEvent.setup()  
const input = screen.getByRole('textbox')  
  
await user.type(input, 'hello{Enter}') // type then press Enter  
await user.type(input, 'hello{Tab}') // type then Tab (moves focus)  
await user.keyboard('{Escape}') // just press Escape  
await user.keyboard('{Control>}a{/Control}') // Ctrl+A (select all)  
await user.keyboard('{Control>}c{/Control}') // Ctrl+C (copy)  
  
// Check focus moved after Tab  
await user.tab()  
expect(screen.getByLabelText(/password/i)).toHaveFocus()
```

5.4 Select Dropdown

```
...  
const user = userEvent.setup()  
const select = screen.getByRole('combobox', { name: /country/i })  
  
await user.selectOptions(select, 'India') // by option text  
await user.selectOptions(select, 'IN') // by option value  
expect(select).toHaveValue('IN')  
  
// Multi-select
```

```
const multiSelect = screen.getByRole('listbox')
await user.selectOptions(multiSelect, ['Option A', 'Option B'])
await user.deselectOptions(multiSelect, 'Option A')
```

5.5 Checkbox and Radio

```
const user = userEvent.setup()

// Checkbox
const checkbox = screen.getByRole('checkbox', { name: /agree/i })
expect(checkbox).not.toBeChecked()
await user.click(checkbox)
expect(checkbox).toBeChecked()
await user.click(checkbox) // toggle off
expect(checkbox).not.toBeChecked()

// Radio
await user.click(screen.getByRole('radio', { name: /yes/i }))
expect(screen.getByRole('radio', { name: /yes/i })).toBeChecked()
expect(screen.getByRole('radio', { name: /no/i })).not.toBeChecked()
```

5.6 File Upload

```
const user = userEvent.setup()
const input = screen.getByLabelText(/upload/i)

// Single file
const file = new File(['file content'], 'test.png', { type: 'image/png' })
await user.upload(input, file)
expect(input.files[0]).toBe(file)
expect(input.files).toHaveLength(1)

// Multiple files
const files = [
  new File(['a'], 'a.png', { type: 'image/png' }),
  new File(['b'], 'b.pdf', { type: 'application/pdf' }),
]
await user.upload(input, files)
expect(input.files).toHaveLength(2)
```

5.7 Hover / Tooltip Test

```
const user = userEvent.setup()
const trigger = screen.getByRole('button', { name: /info/i })

// Hover to show tooltip
await user.hover(trigger)
expect(await screen.findByRole('tooltip')).toBeInTheDocument()

// Unhover to hide it
await user.unhover(trigger)
await waitForElementToBeRemoved(() => screen.queryByRole('tooltip'))
```

5.8 Drag and Drop (fireEvent)

```
// userEvent doesn't support DnD well - use fireEvent
fireEvent.dragStart(draggableEl)
```

```
fireEvent.dragEnter(dropzoneEl)
fireEvent.dragOver(dropzoneEl)
fireEvent.drop(dropzoneEl)
fireEvent.dragEnd(draggableEl)

expect(dropzoneEl).toHaveTextContent('Dropped!')
```

Chapter 6 — Async Patterns — All Scenarios

6.1 Scenario: Loading State → Data

```
...
// Component: shows "Loading..." then data
test('shows data after loading', async () => {
  render(<UserList />)

  // Loading state is immediate
  expect(screen.getByText(/loading/i)).toBeInTheDocument()

  // Wait for data to appear
  await screen.findByRole('list') // or:
  // await waitFor(() => expect(screen.getByRole('list')).toBeInTheDocument())

  // Loading is gone, data is there
  expect(screen.queryByText(/loading/i)).not.toBeInTheDocument()
  expect(screen.getAllByRole('listitem')).toHaveLength(3)
})
```

6.2 Scenario: Spinner → Gone

```
...
// Component uses aria-label="loading..." on Spinner
test('removes spinner after submit', async () => {
  const user = userEvent.setup()
  render(<LoginSubmission />)

  await user.type(screen.getByLabelText(/username/i), 'john')
  await user.type(screen.getByLabelText(/password/i), 'secret')
  await user.click(screen.getByRole('button', { name: /submit/i }))

  // Wait for spinner to disappear (MSW responds instantly in test mode)
  await waitForElementToBeRemoved(screen.getByLabelText(/loading/i))

  // Final state
  expect(screen.getByText('john')).toBeInTheDocument()
})
```

6.3 Scenario: Form Validation Errors

```
...
test('shows field errors on empty submit', async () => {
  const user = userEvent.setup()
  render(<SignupForm />)

  // Submit without filling anything
  await user.click(screen.getByRole('button', { name: /register/i }))

  // Errors appear (may be async if validated on submit)
  await screen.findByText(/email is required/i)
  expect(screen.getByText(/password is required/i)).toBeInTheDocument()

  // Fill email, re-submit
  await user.type(screen.getByLabelText(/email/i), 'bad-email')
  await user.click(screen.getByRole('button', { name: /register/i }))
  expect(await screen.findByText(/invalid email/i)).toBeInTheDocument()
})
```

6.4 Scenario: waitFor — Polling / Retrying

```
// waitFor retries until assertion passes or times out (default 1000ms)
await waitFor(() => {
  expect(screen.getByRole('alert')).toHaveTextContent('Success!')
})

// With custom timeout
await waitFor(
  () => expect(screen.getByText(/data loaded/i)).toBeInTheDocument(),
  { timeout: 5000, interval: 200 }
)

// waitFor on multiple assertions
await waitFor(() => {
  expect(screen.queryByText(/loading/i)).not.toBeInTheDocument()
  expect(screen.getByRole('table')).toBeInTheDocument()
})
```

⚠ WARN

Never use `waitFor` with side effects (clicks). Use `findBy` instead, or await the action first, then `waitFor`.

6.5 Scenario: Debounced Input (Search)

```
// Component: searches after 300ms debounce
test('searches after debounce', async () => {
  const user = userEvent.setup()
  jest.useFakeTimers() // control time

  render(<SearchBox />)
  await user.type(screen.getByRole('textbox'), 'react')

  // Before debounce fires — no results yet
  expect(screen.queryByRole('list')).not.toBeInTheDocument()

  // Advance timers past debounce delay
  act(() => { jest.advanceTimersByTime(300) })

  await screen.findByRole('list')
  expect(screen.getAllByRole('listitem')).toHaveLength(0)

  jest.useRealTimers()
})
```

6.6 Scenario: Polling / Interval Updates

```
test('auto-refreshes data every 5 seconds', async () => {
  jest.useFakeTimers()
  render(<LiveData />)

  expect(screen.getByText('0 users online')).toBeInTheDocument()

  // Simulate 5 seconds passing
  await act(async () => { jest.advanceTimersByTime(5000) })
  expect(await screen.findByText('5 users online')).toBeInTheDocument()

  await act(async () => { jest.advanceTimersByTime(5000) })
```

```
expect(await screen.findByText('3 users online')).toBeInTheDocument()

jest.useRealTimers()
})
```

Chapter 7 — MSW v2 — All Scenarios

7.1 How MSW v2 Works

MSW v2 uses the standard Fetch Request/Response API — same as the browser. Handlers receive a real Request object and return a real Response (via HttpResponse helper).

MSW v2 Concept	Description
http.get / http.post / http.put / http.patch / http.delete	Method-specific interceptors
{ request, params, cookies }	Handler receives destructured args — no more req/res/ctx
HttpResponse.json(body, init?)	Return JSON response — replaces res(ctx.json(...))
HttpResponse.text(body, init?)	Return plain text response
new HttpResponse(body, init)	Return custom response — replaces res(ctx.status(...))
HttpResponse.error()	Simulate network error — replaces res.networkError()
await delay(ms)	Await delay inside handler — replaces ctx.delay()
server.use(...handlers)	Add one-off override handlers for a specific test
server.resetHandlers()	Remove all one-off handlers (call in afterEach)

7.2 Setup Pattern (Per Test File)

```
import { http, HttpResponse } from 'msw'
import { setupServer } from 'msw/node'
import { handlers } from '../test/mocks/handlers'

// Create server with your shared default handlers
const server = setupServer(...handlers)

beforeAll(() => server.listen({ onUnhandledRequest: 'error' }))
// 'error' → fail the test if an unexpected request is made (recommended)
// 'warn' → log it but don't fail
// 'bypass' → let real network through (avoid in tests)

afterEach(() => server.resetHandlers()) // remove per-test overrides
afterAll(() => server.close())
```

7.3 handlers.ts — All Common Patterns

```
// src/test/mocks/handlers.ts
import { http, HttpResponse, delay } from 'msw'

export const handlers = [

  // — GET — returns JSON —————
  http.get('https://api.example.com/users', () => {
    return HttpResponse.json([
      { id: 1, name: 'John' },
      { id: 2, name: 'Jane' },
    ])
  })
],
```

```

// — GET with URL params (:id) —————
http.get('https://api.example.com/users/:id', ({ params }) => {
  const { id } = params // typed string — same as req.params in v1
  return HttpResponse.json({ id: Number(id), name: 'John' })
}),

// — GET with query params —————
http.get('https://api.example.com/search', ({ request }) => {
  const url = new URL(request.url)
  const q = url.searchParams.get('q') || ''
  return HttpResponse.json({ results: [`Result for: ${q}`] })
}),

// — POST — read body —————
http.post('https://api.example.com/login', async ({ request }) => {
  const { username, password } = await request.json()
  if (!password) return HttpResponse.json({ message: 'password required' }, {
    status: 400 })
  if (!username) return HttpResponse.json({ message: 'username required' }, {
    status: 400 })
  return HttpResponse.json({ username })
}),

// — PUT —————
http.put('https://api.example.com/users/:id', async ({ params, request }) => {
  const body = await request.json()
  return HttpResponse.json({ id: Number(params.id), ...body })
}),

// — DELETE —————
http.delete('https://api.example.com/users/:id', ({ params }) => {
  return HttpResponse.json({ deleted: true, id: Number(params.id) })
}),

// — Read request headers —————
http.get('https://api.example.com/me', ({ request }) => {
  const token = request.headers.get('Authorization')
  if (!token) return new HttpResponse(null, { status: 401 })
  return HttpResponse.json({ username: 'john' })
}),
]

```

7.4 Scenario: Happy Path (GET list)

```

...
test('displays list of users', async () => {
  // Default handler already returns [{ id:1, name:'John' }, { id:2, name:'Jane'
  }]
  render(<UserList />)

  const items = await screen.findAllByRole('listitem')
  expect(items).toHaveLength(2)
  expect(items[0]).toHaveTextContent('John')
  expect(items[1]).toHaveTextContent('Jane')
})

```

7.5 Scenario: API Error (500)

```

...
test('shows error on server failure', async () => {
  server.use(
    http.get('https://api.example.com/users', () => {

```

```

        return HttpResponse.json(
          { message: 'Internal Server Error' },
          { status: 500 }
        )
      })
    )

    render(<UserList />)

    await waitForElementToBeRemoved(screen.getByText(/loading/i))
    expect(screen.getByRole('alert')).toHaveTextContent('Internal Server Error')
  })

```

7.6 Scenario: Network Error (offline)

```

test('shows network error when offline', async () => {
  server.use(
    http.get('https://api.example.com/users', () => {
      return HttpResponse.error() // simulates total network failure
    })
  )

  render(<UserList />)

  expect(await screen.findByRole('alert')).toHaveTextContent(/network/i)
})

```

7.7 Scenario: 401 Unauthorized

```

test('redirects to login on 401', async () => {
  server.use(
    http.get('https://api.example.com/profile', () => {
      return new HttpResponse(null, { status: 401 })
    })
  )

  render(<ProfilePage />, { wrapper: MemoryRouter })

  await screen.findByRole('heading', { name: /login/i })
})

```

7.8 Scenario: Slow Response (Loading State)

```

import { http, HttpResponse, delay } from 'msw'

test('shows loading spinner during fetch', async () => {
  server.use(
    http.get('https://api.example.com/users', async () => {
      await delay(500) // wait 500ms before responding
      return HttpResponse.json([
        { id: 1, name: 'John' }
      ])
    })
  )

  render(<UserList />)

  expect(screen.getByLabelText(/loading/i)).toBeInTheDocument()

  await screen.findByRole('listitem')

```

```
expect(screen.queryByLabelText(/loading/i)).not.toBeInTheDocument()
})
```

7.9 Scenario: POST — Capture & Assert Request Body

```
test('sends correct body to API', async () => {
  let capturedBody

  server.use(
    http.post('https://api.example.com/users', async ({ request }) => {
      capturedBody = await request.json() // capture what was sent
      return HttpResponse.json({ id: 99, ...capturedBody }, { status: 201 })
    })
  )

  const user = userEvent.setup()
  render(<CreateUserForm />)

  await user.type(screen.getByLabelText(/name/i), 'Jane')
  await user.type(screen.getByLabelText(/email/i), 'jane@test.com')
  await user.click(screen.getByRole('button', { name: /create/i }))

  await screen.findByText('Jane created!')

  expect(capturedBody).toEqual({ name: 'Jane', email: 'jane@test.com' })
})
```

7.10 Scenario: Pagination (Query Params)

```
test('loads next page on click', async () => {
  server.use(
    http.get('https://api.example.com/users', ({ request }) => {
      const url = new URL(request.url)
      const page = url.searchParams.get('page') || '1'

      if (page === '1') return HttpResponse.json({ users: ['Alice', 'Bob'],
        hasNext: true })
      if (page === '2') return HttpResponse.json({ users: ['Charlie', 'Dave'],
        hasNext: false })
    })
  )

  const user = userEvent.setup()
  render(<PaginatedList />)

  expect(await screen.findByText('Alice')).toBeInTheDocument()
  expect(screen.getByRole('button', { name: /next page/i })).toBeInTheDocument()

  await user.click(screen.getByRole('button', { name: /next page/i }))

  expect(await screen.findByText('Charlie')).toBeInTheDocument()
  expect(screen.queryByRole('button', { name: /next page/i
})).not.toBeInTheDocument()
})
```

7.11 Scenario: Auth Header Check

```
test('passes auth token in request', async () => {
```

```

let receivedToken

server.use(
  http.get('https://api.example.com/me', ({ request }) => {
    receivedToken = request.headers.get('Authorization')
    return HttpResponse.json({ username: 'john' })
  })
)

render(<ProfilePage />)
await screen.findByText('john')

expect(receivedToken).toBe('Bearer test-token-123')
})

```

7.12 Scenario: once() — Override Only One Call

```

import { http, HttpResponse } from 'msw'

test('retries after first failure', async () => {
  let callCount = 0

  server.use(
    http.get('https://api.example.com/data', () => {
      callCount++
      if (callCount === 1) {
        // First call fails
        return HttpResponse.json({ message: 'Service Unavailable' }, { status: 503 })
      }
      // Second call succeeds (retry)
      return HttpResponse.json({ data: 'loaded' })
    })
  )

  render(<DataWithRetry />)

  // Shows error first
  expect(await screen.findByRole('alert')).toBeInTheDocument()

  // After retry button click
  await userEvent.click(screen.getByRole('button', { name: /retry/i }))
  expect(await screen.findByText('loaded')).toBeInTheDocument()
})

```

7.13 Scenario: GraphQL with MSW v2

```

import { graphql, HttpResponse } from 'msw'
import { setupServer } from 'msw/node'

const server = setupServer(
  graphql.query('GetUser', ({ variables }) => {
    return HttpResponse.json({
      data: { user: { id: variables.id, name: 'John' } },
    })
  }),
  graphql.mutation('CreateUser', ({ variables }) => {
    return HttpResponse.json({
      data: { createUser: { id: 1, name: variables.name } },
    })
  })
)

```

```

    })
  }},
  // Error scenario
  graphql.query('GetUser', () => {
    return HttpResponse.json({
      errors: [{ message: 'User not found' }],
    })
  })
)

```

7.14 Browser Dev Mode — setupWorker

```

// src/mocks/browser.ts - for Vite dev mode
import { setupWorker } from 'msw/browser'
import { handlers } from './handlers'

export const worker = setupWorker(...handlers)

// src/main.tsx - start worker before rendering
async function prepare() {
  if (import.meta.env.DEV) {
    const { worker } = await import('./mocks/browser')
    await worker.start({
      onUnhandledRequest: 'bypass', // allow real API calls in dev
      serviceWorker: {
        url: '/mockServiceWorker.js', // from: npx msw init public/
      },
    })
  }
}

prepare().then(() => {
  ReactDOM.createRoot(document.getElementById('root')!).render(<App />)
})

```

NOTE

setupWorker is for browser/dev mode only. setupServer from msw/node is for Jest/Vitest tests running in Node.

7.15 HttpResponse — All Response Types

```

import { HttpResponse } from 'msw'

// JSON (most common)
HttpResponse.json({ data: 'value' })
HttpResponse.json({ error: 'Not found' }, { status: 404 })

// Plain text
HttpResponse.text('Hello world')
HttpResponse.text('Not authorized', { status: 401 })

// No body (204, 401 etc.)
new HttpResponse(null, { status: 204 }) // No Content
new HttpResponse(null, { status: 401 }) // Unauthorized

// Custom headers
HttpResponse.json({ data: 'ok' }, {
  status: 200,
  headers: {

```

```
      'X-Custom-Header': 'value',
      'Set-Cookie': 'session=abc123',
    },
  })

  // Network failure (no response reaches client)
  HttpResponse.error()

  // Delayed response
  import { delay } from 'msw'
  async () => {
    await delay(1000) // real milliseconds delay
    await delay('infinite') // never resolves (loading forever test)
    return HttpResponse.json({ data: 'late' })
  }
}
```

Chapter 8 — Mocking Patterns — All Scenarios

8.1 jest.fn() / vi.fn() — All Patterns

```
// Create
const fn = jest.fn()
const fn = jest.fn(() => 42) // with default
implementation
const fn = jest.fn().mockReturnValue(42) // same
const fn = jest.fn().mockResolvedValue({ id: 1 }) // async success
const fn = jest.fn().mockRejectedValue(new Error()) // async failure

// Per-call control
fn.mockReturnValueOnce('first').mockReturnValueOnce('second')
// first call → 'first', second call → 'second', subsequent → undefined

// Assert on calls
expect(fn).toHaveBeenCalled()
expect(fn).toHaveBeenCalledTimes(2)
expect(fn).toHaveBeenCalledWith('arg1', { key: 'val' })
expect(fn).toHaveBeenLastCalledWith('last-arg')
expect(fn).toHaveBeenNthCalledWith(1, 'first-call-arg')

// Inspect calls manually
fn.mock.calls // [[arg1, arg2], [arg1], ...]
fn.mock.calls[0] // first call args array
fn.mock.calls[0][0] // first arg of first call
fn.mock.results // [{ type: 'return', value: 42 }, ...]
fn.mock.results[0].value // return value of first call

// Reset
fn.mockClear() // clears .mock (calls/results) — keeps implementation
fn.mockReset() // clears .mock + implementation
fn.mockRestore() // restores original (only for spies)
```

8.2 jest.mock() Module Mocking

```
// — 1. Auto-mock (all exports become jest.fn()) —
jest.mock('./api')
// Every exported function is now jest.fn()

// — 2. Factory mock —
jest.mock('./api', () => ({
  getUser: jest.fn().mockResolvedValue({ id: 1, name: 'John' }),
  createUser: jest.fn().mockResolvedValue({ id: 2 }),
}))

// — 3. Partial mock (keep some real) —
jest.mock('./utils', () => {
  const actual = jest.requireActual('./utils') // import real module
  return {
    ...actual,
    formatDate: jest.fn().mockReturnValue('2024-01-01'), // only mock this
  }
})

// — 4. Mock ES default export —
jest.mock('./useTheme', () => ({
  __esModule: true,
  default: jest.fn().mockReturnValue(['light', jest.fn()]),
}))
```

```

}))

// — 5. Reset mock implementation per test —————
import { getUser } from './api' // after jest.mock, this is jest.fn()

beforeEach(() => {
  getUser.mockResolvedValue({ id: 1, name: 'Default' })
})

test('shows error', () => {
  getUser.mockRejectedValueOnce(new Error('Not found'))
  // ...
})

```

8.3 vi.mock() — Vitest Equivalent

```

// Same as jest.mock but:
// 1. vi.importActual is ASYNC
// 2. Factory can be async

vi.mock('./api', async () => {
  const actual = await vi.importActual('./api') // ← async
  return {
    ...actual,
    getUser: vi.fn().mockResolvedValue({ id: 1 }),
  }
})

```

8.4 jest.spyOn — Spy on Real Methods

```

// Spy on object method without replacing it (records calls)
const spy = jest.spyOn(Math, 'random')
Math.random() // original still called
expect(spy).toHaveBeenCalled()
spy.mockRestore()

// Spy + replace implementation
const spy = jest.spyOn(console, 'error').mockImplementation(() => {})
// original NOT called — just intercepted

// Spy on imported module function
import * as api from './api'
const spy = jest.spyOn(api, 'fetchUser').mockResolvedValue({ id: 42 })

// Common: suppress noisy console during tests
beforeEach(() => {
  jest.spyOn(console, 'error').mockImplementation(() => {})
})
afterEach(() => jest.restoreAllMocks())

```

8.5 Mocking window / browser APIs

```

// Scenario: localStorage
const mockLocalStorage = () => {
  let store = {}
  return {
    getItem: jest.fn(key => store[key] || null),
    setItem: jest.fn((key, val) => { store[key] = String(val) }),
    removeItem: jest.fn(key => { delete store[key] }),
  }
}

```

```

    clear: jest.fn(() => { store = {} })),
  }
})()
Object.defineProperty(window, 'localStorage', { value: mockLocalStorage })

// Scenario: window.location
delete window.location
window.location = { href: '', assign: jest.fn(), reload: jest.fn() }

// Scenario: matchMedia
window.matchMedia = jest.fn().mockImplementation(query => ({
  matches: false,
  media: query,
  onchange: null,
  addListener: jest.fn(),
  removeListener: jest.fn(),
  addEventListener: jest.fn(),
  removeEventListener: jest.fn(),
  dispatchEvent: jest.fn(),
}))

// Scenario: IntersectionObserver
const mockObserver = { observe: jest.fn(), unobserve: jest.fn(), disconnect:
jest.fn() }
window.IntersectionObserver = jest.fn(() => mockObserver)

// Scenario: geolocation
window.navigator.geolocation = {
  getCurrentPosition: jest.fn(),
  watchPosition: jest.fn(),
  clearWatch: jest.fn(),
}

```

8.6 Mocking Timers

```

// Enable fake timers
jest.useFakeTimers() // or vi.useFakeTimers()

// Control time
jest.advanceTimersByTime(1000) // advance 1 second
jest.runAllTimers() // run all pending timers immediately
jest.runOnlyPendingTimers() // run currently pending (not newly scheduled)
jest.clearAllTimers() // remove all timers

// Modern API (Jest 27+ / Vitest)
jest.advanceTimersByTimeAsync(1000) // for async timers

// Real usage — debounce test
jest.useFakeTimers()
const user = userEvent.setup({ delay: null }) // instant typing
render(<SearchBox />)
await user.type(screen.getByRole('textbox'), 'react')
act(() => jest.advanceTimersByTime(300)) // fire debounce
await screen.findByRole('list')
jest.useRealTimers()

```

Chapter 9 — Component Testing — Real Scenarios

9.1 Testing a Modal

```
test('opens and closes modal', async () => {
  const user = userEvent.setup()
  render(<App />)

  // Modal initially not visible
  expect(screen.queryByRole('dialog')).not.toBeInTheDocument()

  // Open modal
  await user.click(screen.getByRole('button', { name: /open modal/i }))
  expect(screen.getByRole('dialog')).toBeInTheDocument()
  expect(screen.getByRole('heading', { name: /confirm/i })).toBeInTheDocument()

  // Close via close button
  await user.click(screen.getByRole('button', { name: /close/i }))
  await waitForElementToBeRemoved(screen.queryByRole('dialog'))
})
```

9.2 Testing a Tab Component

```
test('switches tabs', async () => {
  const user = userEvent.setup()
  render(<TabPanel />)

  // Tab 1 content visible, Tab 2 hidden
  expect(screen.getByRole('tabpanel')).toHaveTextContent('Tab 1 content')

  // Click Tab 2
  await user.click(screen.getByRole('tab', { name: /tab 2/i }))

  // Tab 2 content now visible
  expect(screen.getByRole('tabpanel')).toHaveTextContent('Tab 2 content')

  // Tab 2 is selected (aria-selected)
  expect(screen.getByRole('tab', { name: /tab 2/i })).toHaveAttribute('aria-selected', 'true')
})
```

9.3 Testing a Dropdown / Autocomplete

```
test('filters options on type', async () => {
  const user = userEvent.setup()
  render(<Autocomplete options={['React', 'Redux', 'Router']} />)

  const input = screen.getByRole('combobox')
  await user.type(input, 'Re')

  // Dropdown appears with filtered results
  const options = await screen.findAllByRole('option')
  expect(options).toHaveLength(2)
  expect(options[0]).toHaveTextContent('React')
  expect(options[1]).toHaveTextContent('Redux')

  // Select an option
```

```

    await user.click(options[0])
    expect(input).toHaveValue('React')
    expect(screen.queryByRole('listbox')).not.toBeInTheDocument()
  })
}

```

9.4 Testing a Multi-Step Form

```

test('completes multi-step form', async () => {
  const user = userEvent.setup()
  const onComplete = jest.fn()
  render(<MultiStepForm onComplete={onComplete} />)

  // Step 1
  expect(screen.getByText(/step 1 of 3/i)).toBeInTheDocument()
  await user.type(screen.getByLabelText(/name/i), 'John')
  await user.click(screen.getByRole('button', { name: /next/i }))

  // Step 2
  expect(screen.getByText(/step 2 of 3/i)).toBeInTheDocument()
  await user.type(screen.getByLabelText(/email/i), 'john@test.com')
  await user.click(screen.getByRole('button', { name: /next/i }))

  // Step 3 - review
  expect(screen.getByText('John')).toBeInTheDocument()
  expect(screen.getByText('john@test.com')).toBeInTheDocument()
  await user.click(screen.getByRole('button', { name: /submit/i }))

  expect(onComplete).toHaveBeenCalledWith({ name: 'John', email: 'john@test.com' })
})

```

9.5 Testing Error Boundaries

```

import { ErrorBoundary } from 'react-error-boundary'

const ThrowError = () => { throw new Error('Kaboom!') }

test('shows fallback on error', () => {
  // Suppress React's error output for this test
  const spy = jest.spyOn(console, 'error').mockImplementation(() => {})

  render(
    <ErrorBoundary fallback={<div role="alert">Something went wrong</div>}>
      <ThrowError />
    </ErrorBoundary>
  )

  expect(screen.getByRole('alert')).toHaveTextContent('Something went wrong')
  spy.mockRestore()
})

```

9.6 Testing with Context (Theme)

```

// src/test/test-utils.tsx - custom render wrapping all providers
import { render as rtlRender } from '@testing-library/react'
import { ThemeProvider } from '../context/ThemeContext'
import { AuthProvider } from '../context/AuthContext'
import { MemoryRouter } from 'react-router-dom'
import { Provider } from 'react-redux'

```

```
import { store } from '../store'

function AllProviders({ children }) {
  return (
    <Provider store={store}>
      <MemoryRouter>
        <AuthProvider>
          <ThemeProvider>{children}</ThemeProvider>
        </AuthProvider>
      </MemoryRouter>
    </Provider>
  )
}

export function render(ui, options = {}) {
  return rtlRender(ui, { wrapper: AllProviders, ...options })
}
export * from '@testing-library/react'

// In test file
import { render, screen } from '../test/test-utils'
import EasyButton from '../components/EasyButton'

test('renders with dark theme', () => {
  render(<EasyButton>Click</EasyButton>)
  expect(screen.getByRole('button')).toHaveStyle('background-color: black')
})
```

9.7 Testing React Router Navigation

```
import { MemoryRouter, Routes, Route } from 'react-router-dom'

function renderWithRouter(ui, { initialEntries = ['/'] } = {}) {
  return render(
    <MemoryRouter initialEntries={initialEntries}>{ui}</MemoryRouter>
  )
}

test('navigates on link click', async () => {
  const user = userEvent.setup()
  renderWithRouter(
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/about" element={<About />} />
    </Routes>
  )

  await user.click(screen.getByRole('link', { name: /about/i }))
  expect(screen.getByRole('heading', { name: /about us/i })).toBeInTheDocument()
})

// Test current route
test('highlights active nav link', () => {
  renderWithRouter(<Nav />, { initialEntries: ['/about'] })
  expect(screen.getByRole('link', { name: /about/i })).toHaveClass('active')
})
```

9.8 Testing Infinite Scroll

```
test('loads more on scroll to bottom', async () => {
```

```
// Mock IntersectionObserver
let observerCallback
window.IntersectionObserver = jest.fn(cb => {
  observerCallback = cb
  return { observe: jest.fn(), unobserve: jest.fn(), disconnect: jest.fn() }
})

render(<InfiniteList />)

// Initial items
expect(await screen.findAllByRole('listitem')).toHaveLength(10)

// Simulate scroll to bottom (IntersectionObserver fires)
act(() => {
  observerCallback([{ isIntersecting: true }])
})

// More items loaded
await waitFor(() => {
  expect(screen.getAllByRole('listitem')).toHaveLength(20)
})
})
```

Chapter 10 — Custom Hook Testing — All Scenarios

10.1 renderHook — Full API

```
import { renderHook, act } from '@testing-library/react'

const { result, rerender, unmount } = renderHook(
  (props) => useMyHook(props),
  {
    initialProps: { count: 0 },
    wrapper: ({ children }) => <Provider>{children}</Provider>,
  }
)

result.current // current return value of the hook
rerender({ count: 5 }) // call hook again with new props
unmount() // trigger cleanup (useEffect cleanup runs)
```

10.2 Scenario: Data-Fetching Hook

```
// useFetchUser hook
function useFetchUser(id) {
  const [user, setUser] = React.useState(null)
  const [loading, setLoading] = React.useState(true)
  const [error, setError] = React.useState(null)

  React.useEffect(() => {
    fetch(`/api/users/${id}`)
      .then(r => r.json())
      .then(data => { setUser(data); setLoading(false) })
      .catch(err => { setError(err); setLoading(false) })
  }, [id])

  return { user, loading, error }
}

// Test
test('fetches user by id', async () => {
  const { result } = renderHook(() => useFetchUser(1))

  // Initially loading
  expect(result.current.loading).toBe(true)
  expect(result.current.user).toBeNull()

  // Wait for data
  await waitFor(() => expect(result.current.loading).toBe(false))

  expect(result.current.user).toEqual({ id: 1, name: 'John' })
  expect(result.current.error).toBeNull()
})
```

10.3 Scenario: Hook with Side Effects / Timers

```
// useCountdown hook
function useCountdown(seconds) {
  const [remaining, setRemaining] = React.useState(seconds)
  React.useEffect(() => {
```

```

    if (remaining <= 0) return
    const id = setInterval(() => setRemaining(r => r - 1), 1000)
    return () => clearInterval(id)
  }, [remaining])
  return remaining
}

test('counts down to zero', () => {
  jest.useFakeTimers()

  const { result } = renderHook(() => useCountdown(3))
  expect(result.current).toBe(3)

  act(() => jest.advanceTimersByTime(1000))
  expect(result.current).toBe(2)

  act(() => jest.advanceTimersByTime(2000))
  expect(result.current).toBe(0)

  jest.useRealTimers()
})

```

10.4 Scenario: Hook Cleanup (unmount)

```

test('clears subscription on unmount', () => {
  const unsubscribe = jest.fn()
  const subscribe = jest.fn().mockReturnValue(unsubscribe)

  const { unmount } = renderHook(() => useSubscription(subscribe))

  expect(subscribe).toHaveBeenCalledTimes(1)
  expect(unsubscribe).not.toHaveBeenCalled()

  unmount() // triggers useEffect cleanup

  expect(unsubscribe).toHaveBeenCalledTimes(1)
})

```

10.5 Scenario: Hook that uses Context

```

// useTheme hook requires ThemeProvider
const { result } = renderHook(
  () => useTheme(),
  {
    wrapper: ({ children }) => (
      <ThemeProvider initialTheme="dark">{children}</ThemeProvider>
    )
  }
)

expect(result.current[0]).toBe('dark') // [theme, setTheme]

// Change theme
act(() => result.current[1]('light')) // call setTheme
expect(result.current[0]).toBe('light')

```

Chapter 11 — Snapshot Testing

11.1 When to Use Snapshots

Snapshots catch UNINTENDED changes. Best for:

- Stable, "dumb" presentational components
- Error messages or alerts
- Complex rendered output you want to freeze

Avoid for:

- Highly dynamic content
- Components that change often
- Using snapshots as your ONLY tests

11.2 Component Snapshot

```
test('renders button correctly', () => {
  const { asFragment } = render(<Button variant="primary">Submit</Button>)
  expect(asFragment()).toMatchSnapshot()
})

// Creates: __snapshots__/Button.test.js.snap
// exports['renders button correctly 1'] = `
// <DocumentFragment>
//   <button class="btn btn-primary">Submit</button>
// </DocumentFragment>
// `;
```

11.3 Inline Snapshot

```
// Jest writes the value directly into your test file
test('error message format', () => {
  expect(getErrorMessage(400)).toMatchInlineSnapshot(`"Bad Request"`)
  expect(getErrorMessage(404)).toMatchInlineSnapshot(`"Not Found"`)
  expect(getErrorMessage(500)).toMatchInlineSnapshot(`"Internal Server Error"`)
})
```

11.4 Update Snapshots

```
# Update all snapshots
npx jest -u
npx jest --updateSnapshot

# Update only specific file
npx jest Button.test.js -u
```

NOTE

Always review snapshot diffs before committing. Snapshots are only useful if you check them carefully when they change.

11.5 Snapshot with Dynamic Values

```
// Problem: date/id values change every run
expect(asFragment()).toMatchSnapshot() // ✗ fails because Date.now() changes

// Solution 1: mock dynamic values
jest.spyOn(Date, 'now').mockReturnValue(1699999999999)

// Solution 2: use expect.any() with toMatchObject
expect(result).toMatchObject({
  name: 'John',
  id: expect.any(Number), // don't care about exact id
  createdAt: expect.any(String), // don't care about exact date
})
```

Chapter 12 — All Matchers Reference

12.1 Jest / Vitest Built-in Matchers

Equality

```
• • •
expect(2 + 2).toBe(4) // strict === (primitives)
expect({ a: 1 }).toEqual({ a: 1 }) // deep equality (objects)
expect({ a: 1 }).toStrictEqual({ a: 1 }) // deep + no extra undefined props
```

Truthiness

```
• • •
expect(1).toBeTruthy() // any truthy value
expect('').toBeFalsy() // any falsy value
expect(null).toBeNull()
expect(undefined).toBeUndefined()
expect('hello').toBeDefined()
expect(NaN).toBeNaN()
```

Numbers

```
• • •
expect(5).toBeGreaterThan(4)
expect(5).toBeGreaterThanOrEqual(5)
expect(3).toBeLessThan(4)
expect(3).toBeLessThanOrEqual(3)
expect(0.1 + 0.2).toBeCloseTo(0.3, 5) // floating point
```

Strings

```
• • •
expect('hello world').toMatch('world') // contains substring
expect('hello world').toMatch(/world/) // regex
expect('hello world').toContain('hello')
```

Arrays & Objects

```
• • •
expect([1, 2, 3]).toContain(2) // primitive in array
expect([{id:1}]).toContainEqual({id:1}) // object in array (deep)
expect([1, 2, 3]).toHaveLength(3)
expect({ a:1, b:2 }).toHaveProperty('a')
expect({ a:1, b:2 }).toHaveProperty('a', 1) // with value
expect({ a:1, b:2 }).toMatchObject({ a:1 }) // subset match
```

Errors

```
• • •
expect(() => JSON.parse('bad')).toThrow()
expect(() => JSON.parse('bad')).toThrow(SyntaxError)
expect(() => JSON.parse('bad')).toThrow('Unexpected token')
```

Async

```
• • •
await expect(Promise.resolve(42)).resolves.toBe(42)
await expect(Promise.reject(new Error('fail'))).rejects.toThrow('fail')
await expect(api.getUser(1)).resolves.toMatchObject({ id: 1 })
```

Mock assertions

```
expect(fn).toHaveBeenCalled()
expect(fn).toHaveBeenCalledTimes(2)
expect(fn).toHaveBeenCalledWith('arg1', 'arg2')
expect(fn).toHaveBeenLastCalledWith('last')
expect(fn).toHaveBeenNthCalledWith(2, 'second call arg')
expect(fn).toHaveReturned()
expect(fn).toHaveReturnedWith(42)
expect(fn).not.toHaveBeenCalled()
```

12.2 @testing-library/jest-dom Matchers

```
// DOM presence
expect(el).toBeInTheDocument()
expect(el).not.toBeInTheDocument()

// Content
expect(el).toHaveTextContent('Submit')
expect(el).toHaveTextContent(/submit/i) // regex ok

// Style
expect(el).toHaveStyle('color: red')
expect(el).toHaveStyle({ color: 'red', display: 'flex' })

// Attributes
expect(el).toHaveAttribute('type', 'submit')
expect(el).not.toHaveAttribute('disabled')

// Form
expect(input).toHaveValue('john@test.com')
expect(checkbox).toBeChecked()
expect(input).toBeRequired()
expect(select).toHaveDisplayValue('India')

// Visibility / state
expect(el).toBeVisible()
expect(el).toBeDisabled()
expect(el).toBeEnabled()
expect(input).toHaveFocus()

// Class
expect(el).toHaveClass('active')
expect(el).toHaveClass('btn', 'btn-primary') // multiple
expect(el).not.toHaveClass('disabled')

// Structure
expect(el).toBeEmptyDOMElement()
expect(parent).toContainElement(child)
expect(el).toHaveDescription('Detailed description')
```

Chapter 13 — Advanced Patterns

13.1 Testing Redux Toolkit

```
import { configureStore } from '@reduxjs/toolkit'
import { Provider } from 'react-redux'
import counterReducer, { increment } from './counterSlice'
import Counter from './Counter'

// Create fresh store per test to avoid leakage
function renderWithStore(ui, { preloadedState = {}, store = null, ...options } = {}) {
  const testStore = store || configureStore({
    reducer: { counter: counterReducer },
    preloadedState,
  })
  return {
    store: testStore,
    ...render(<Provider store={testStore}>{ui}</Provider>, options)
  }
}

test('displays initial count from store', () => {
  renderWithStore(<Counter />, { preloadedState: { counter: { value: 10 } } })
  expect(screen.getByText(/10/)).toBeInTheDocument()
})

test('dispatches increment action', async () => {
  const user = userEvent.setup()
  const { store } = renderWithStore(<Counter />)

  await user.click(screen.getByRole('button', { name: /increment/i }))

  expect(store.getState().counter.value).toBe(1)
})
```

13.2 Testing RTK Query

```
import { setupApiStore } from '../test/rtk-test-utils'
import { userApi } from './userApi'
import UserCard from './UserCard'

// RTK Query helper — creates isolated API store for tests
function renderWithApiStore(ui) {
  const { store } = setupApiStore(userApi)
  return render(<Provider store={store}>{ui}</Provider>)
}

test('shows user data', async () => {
  // MSW intercepts the RTK Query fetch
  renderWithApiStore(<UserCard userId={1} />)

  expect(screen.getByText(/loading/i)).toBeInTheDocument()
  expect(await screen.findByText('John Smith')).toBeInTheDocument()
})
```

13.3 Testing React Query (TanStack Query)

```

import { QueryClient, QueryClientProvider } from '@tanstack/react-query'

// Create fresh client per test - prevents cache leakage
function renderWithQuery(ui) {
  const queryClient = new QueryClient({
    defaultOptions: {
      queries: {
        retry: false,    // don't retry in tests
        staleTime: 0,    // always refetch
      },
    },
  })
  return render(
    <QueryClientProvider client={queryClient}>{ui}</QueryClientProvider>
  )
}

test('shows user list', async () => {
  renderWithQuery(<UserList />)

  expect(screen.getByText(/loading/i)).toBeInTheDocument()
  await screen.findByText('John')
  expect(screen.queryByText(/loading/i)).not.toBeInTheDocument()
})

test('shows error state', async () => {
  server.use(http.get('/api/users', () => new HttpResponse(null, { status: 500 })))

  renderWithQuery(<UserList />)

  expect(await screen.findByRole('alert')).toHaveTextContent(/something went wrong/i)
})

```

13.4 Testing Formik Forms

```

test('validates and submits formik form', async () => {
  const user = userEvent.setup()
  const onSubmit = jest.fn()
  render(<SignupForm onSubmit={onSubmit} />)

  // Submit empty - validation triggers
  await user.click(screen.getByRole('button', { name: /submit/i }))
  expect(await screen.findByText(/email is required/i)).toBeInTheDocument()

  // Fill valid data
  await user.type(screen.getByLabelText(/email/i), 'test@example.com')
  await user.type(screen.getByLabelText(/password/i), 'SecurePass123')
  await user.click(screen.getByRole('button', { name: /submit/i }))

  await waitFor(() => {
    expect(onSubmit).toHaveBeenCalledWith(
      { email: 'test@example.com', password: 'SecurePass123' },
      expect.any(Object) // formik helpers object
    )
  })
})

```

13.5 Testing Portals

```

// React Portals render into document.body by default
// RTL's screen queries document.body - so portals work automatically!

test('modal portal renders in body', () => {
  render(<App />)
  userEvent.click(screen.getByRole('button', { name: /open/i }))

  // screen queries body - modal (portaled) is found
  expect(screen.getByRole('dialog')).toBeInTheDocument()
  expect(document.body).toContainElement(screen.getByRole('dialog'))
})

```

13.6 Testing Lazy-loaded Components

```

import { Suspense } from 'react'

const LazyComponent = React.lazy(() => import('./HeavyComponent'))

test('renders after lazy load', async () => {
  render(
    <Suspense fallback=<div>Loading...</div>>
      <LazyComponent />
    </Suspense>
  )

  expect(screen.getByText('Loading...')).toBeInTheDocument()

  // Wait for lazy component to load
  expect(await screen.findByRole('main')).toBeInTheDocument()
  expect(screen.queryByText('Loading...')).not.toBeInTheDocument()
})

```

Chapter 14 — Interview Cheat Sheet

14.1 Top 15 Questions

Question	Concise Answer
RTL guiding principle?	"Tests should resemble how users use the software" — query by role, label, text — never by DOM structure
getBy vs queryBy vs findBy?	getBy: throws if not found queryBy: returns null findBy: async Promise
When to use queryBy?	Only for asserting element does NOT exist: expect(queryBy...).not.toBeInTheDocument()
fireEvent vs userEvent?	fireEvent fires 1 event synchronously. userEvent fires full realistic chain (focus+keydown+keyup+etc) and is async.
What is act()?	Flushes pending React state updates before assertions. RTL wraps it for you automatically. Needed manually in renderHook or raw tests.
How to test async UI?	findBy* queries (findByText, findByRole), waitFor(), waitForElementToBeRemoved()
What is MSW?	Mock Service Worker — intercepts HTTP at network level, not in-code. Works in Node (tests) and browser (dev mode).
When to use server.use() in tests?	To override default handlers for specific error/edge cases. Resets via afterEach(resetHandlers).
How to test components with Context?	Wrap in Provider, or pass wrapper option to render(), or create custom render() in test-utils.
How to test custom hooks?	Use renderHook() from RTL. Returns { result, rerender, unmount }. Access value via result.current.
Jest vs Vitest?	Vitest is faster (Vite ESM-native), vi instead of jest, config in vite.config.ts. Near-identical API.
What is identity-obj-proxy?	A mock for CSS Modules in Jest — returns className as-is so tests don't fail on CSS imports.
Why avoid testId?	It's implementation detail — won't catch real accessibility issues. Use role/label/text instead.
What does renderHook rerender do?	Calls the hook again with new props — useful for testing prop changes without re-mounting.
How to suppress console.error in tests?	jest.spyOn(console,"error").mockImplementation(()=>{}) — restore after with mockRestore()

14.2 Complete Import Quick Reference

```
// — React Testing Library —
import { render, screen, fireEvent, act,
  waitFor, waitForElementToBeRemoved,
  renderHook, cleanup } from '@testing-library/react'

// — User Events —
import userEvent from '@testing-library/user-event'
```

```

const user = userEvent.setup()

// — Jest DOM (in setup file) —
import '@testing-library/jest-dom'

// — MSW v2 —
import { http, HttpResponse, delay, graphql } from 'msw'
import { setupServer } from 'msw/node'
import { setupWorker } from 'msw/browser' // browser/dev mode only

// — Test Data —
import { build, fake, sequence, oneOf } from '@jackfranklin/test-data-bot'
import { faker } from '@faker-js/faker'

// — Jest globals (or import from @jest/globals) —
// describe, test, it, expect, jest
// beforeEach, afterEach, beforeEach, afterEach

// — Vitest —
import { describe, test, expect, vi,
  beforeEach, afterEach, beforeEach, afterEach } from 'vitest'

```

14.3 Standard Test Template

```

// ComponentName.test.tsx
import { render, screen, waitForElementToBeRemoved } from '@testing-library/react'
import userEvent from '@testing-library/user-event'
import { http, HttpResponse } from 'msw'
import { setupServer } from 'msw/node'
import { build, fake } from '@jackfranklin/test-data-bot'
import ComponentName from './ComponentName'

// — MSW —
const server = setupServer(
  http.get('/api/data', () => HttpResponse.json({ items: [] }))
)
beforeAll(() => server.listen())
afterEach(() => server.resetHandlers())
afterAll(() => server.close())

// — Test data factory —
const buildFormData = build({
  fields: {
    email: fake(f => f.internet.email()),
    name: fake(f => f.person.fullName()),
  },
})

// — Tests —
describe('ComponentName', () => {
  test('renders initial state', () => {
    render(<ComponentName />)
    expect(screen.getByRole('heading')).toBeInTheDocument()
  })

  test('handles user interaction', async () => {
    const user = userEvent.setup()
    const onAction = jest.fn()
    render(<ComponentName onAction={onAction} />)

    await user.click(screen.getByRole('button', { name: /action/i }))
    expect(onAction).toHaveBeenCalledTimes(1)
  })
})

```

```

}))

test('shows data after loading', async () => {
  render(<ComponentName />)
  await waitForElementToBeRemoved(screen.getByText(/loading/i))
  expect(screen.getByRole('list')).toBeInTheDocument()
})

test('shows error on API failure', async () => {
  server.use(http.get('/api/data', () => new HttpResponse(null, { status: 500
})))
  render(<ComponentName />)
  expect(await screen.findByRole('alert')).toBeInTheDocument()
})
})

```

14.4 Common Errors & Fixes

Error / Warning	Fix
"not wrapped in act(...)" warning	Wrap state-triggering code in act(), or use waitFor/findBy to wait for async updates to complete
"Unable to find role button"	Call screen.debug() to see actual DOM. Check button text matches name option. Ensure component rendered.
Test passes in isolation, fails in suite	Mock state leaking. Use beforeEach to reset — jest.clearAllMocks(), server.resetHandlers()
"password" input not found by textbox role	Password input has no textbox role. Use getByLabelText(/password/i) instead.
MSW handler not intercepting	URL must match exactly including protocol. Check full URL including https:// and domain.
Context error "must be used inside Provider"	Use wrapper option in render/renderHook, or custom render from test-utils
"toHaveStyle" not matching	Style must match exactly. Computed styles only work in real browsers. Use inline styles in components for reliable tests.
findBy times out	Element never appears. Call screen.debug() to see state. Check MSW handler returns correct data.
Snapshot keeps changing	Dynamic values (Date, ID, random). Mock them with jest.spyOn or use toMatchObject with expect.any()
import.meta.env not defined in Jest	Jest doesn't support import.meta. Use process.env or Vitest instead for Vite projects.

Chapter 15 — Testing Utility Functions

Utility functions are the easiest and most valuable tests to write. No DOM, no rendering — pure input → output. They run instantly and give extremely high confidence.

15.1 What Counts as a Utility?

- Formatters — `formatDate`, `formatCurrency`, `formatPhoneNumber`
- Validators — `isValidEmail`, `isStrongPassword`, `isValidPAN`
- Transformers — `mapApiResponse`, `normalizeUser`, `flattenTree`
- Calculators — `calculateTax`, `getDiscountedPrice`, `paginate`
- String helpers — `truncate`, `slugify`, `capitalize`, `sanitizeHtml`
- Array/Object helpers — `groupBy`, `sortBy`, `deepMerge`, `pick`, `omit`
- Auth helpers — `parseJwt`, `hasPermission`, `isTokenExpired`
- API helpers — `buildQueryString`, `parseErrorResponse`, `retry`

15.2 Formatters

```
// src/utils/format.ts
export function formatCurrency(amount: number, currency = 'INR'): string {
  return new Intl.NumberFormat('en-IN', { style: 'currency', currency
}).format(amount)
}

export function formatDate(date: Date | string, pattern = 'DD MMM YYYY'): string {
  const d = new Date(date)
  if (isNaN(d.getTime())) return 'Invalid date'
  const day = String(d.getDate()).padStart(2, '0')
  const month = d.toLocaleString('en', { month: 'short' })
  const year = d.getFullYear()
  return `${day} ${month} ${year}`
}

export function truncate(str: string, maxLength: number): string {
  if (str.length <= maxLength) return str
  return str.slice(0, maxLength - 3) + '...'
}

export function slugify(str: string): string {
  return str.toLowerCase().trim().replace(/[a-z0-9]+/g, '-').replace(/^-|-$/g,
)
}

// src/utils/format.test.ts
import { formatCurrency, formatDate, truncate, slugify } from './format'

describe('formatCurrency', () => {
  test('formats INR correctly', () => {
    expect(formatCurrency(1000)).toBe('₹1,000.00')
    expect(formatCurrency(100000)).toBe('₹1,00,000.00')
  })

  test('formats USD when passed', () => {
    expect(formatCurrency(1000, 'USD')).toBe('$1,000.00')
  })
})
```

```

    test('handles zero', () => {
      expect(formatCurrency(0)).toBe('₹0.00')
    })

    test('handles negative amounts', () => {
      expect(formatCurrency(-500)).toBe('-₹500.00')
    })
  })

  describe('formatDate', () => {
    test('formats valid date', () => {
      expect(formatDate(new Date('2024-01-15'))).toBe('15 Jan 2024')
    })

    test('formats string date', () => {
      expect(formatDate('2024-06-01')).toBe('01 Jun 2024')
    })

    test('returns Invalid date for bad input', () => {
      expect(formatDate('not-a-date')).toBe('Invalid date')
    })
  })

  describe('truncate', () => {
    test('does not truncate if short enough', () => {
      expect(truncate('hello', 10)).toBe('hello')
    })

    test('truncates with ellipsis', () => {
      expect(truncate('hello world', 8)).toBe('hello...')
    })

    test('handles exact length', () => {
      expect(truncate('hello', 5)).toBe('hello')
    })
  })

  describe('slugify', () => {
    test('lowercases and replaces spaces', () => {
      expect(slugify('Hello World')).toBe('hello-world')
    })

    test('removes special characters', () => {
      expect(slugify('React & Redux!')).toBe('react-redux')
    })

    test('trims leading/trailing hyphens', () => {
      expect(slugify(' hello ')).toBe('hello')
    })
  })
})

```

15.3 Validators

```

// src/utils/validators.ts
export function isValidEmail(email: string): boolean {
  return /^[^s@]+@[^s@]+.[^s@]+$/.test(email)
}

export function isValidPhone(phone: string): boolean {
  return /^[6-9]d{9}$/.test(phone.replace(/s/g, ''))
}

```

```

export function isStrongPassword(password: string): { valid: boolean; errors:
string[] } {
  const errors: string[] = []
  if (password.length < 8) errors.push('At least 8 characters')
  if (!/[A-Z]/.test(password)) errors.push('At least one uppercase letter')
  if (!/[0-9]/.test(password)) errors.push('At least one number')
  if (!/^[a-zA-Z0-9]/.test(password)) errors.push('At least one special
character')
  return { valid: errors.length === 0, errors }
}

```

...

```

// src/utils/validators.test.ts
import { isValidEmail, isValidPhone, isStrongPassword } from './validators'

describe('isValidEmail', () => {
  test.each([
    ['valid@example.com', true],
    ['user.name+tag@domain.co.in', true],
    ['invalid', false],
    ['missing@', false],
    ['@nodomain.com', false],
    ['', false],
  ])('isValidEmail(%s) → %s', (email, expected) => {
    expect(isValidEmail(email)).toBe(expected)
  })
})

describe('isValidPhone', () => {
  test('valid Indian numbers', () => {
    expect(isValidPhone('9876543210')).toBe(true)
    expect(isValidPhone('6789012345')).toBe(true)
  })

  test('invalid numbers', () => {
    expect(isValidPhone('1234567890')).toBe(false) // doesn't start with 6-9
    expect(isValidPhone('98765')).toBe(false) // too short
  })
})

describe('isStrongPassword', () => {
  test('accepts strong password', () => {
    expect(isStrongPassword('Secure@123').valid).toBe(true)
    expect(isStrongPassword('Secure@123').errors).toHaveLength(0)
  })

  test('returns all errors for weak password', () => {
    const { valid, errors } = isStrongPassword('weak')
    expect(valid).toBe(false)
    expect(errors).toContain('At least 8 characters')
    expect(errors).toContain('At least one uppercase letter')
    expect(errors).toContain('At least one number')
    expect(errors).toContain('At least one special character')
  })

  test('returns specific error', () => {
    const { errors } = isStrongPassword('StrongPass1') // missing special char
    expect(errors).toHaveLength(1)
    expect(errors[0]).toMatch(/special character/i)
  })
})

```

15.4 Transformers / Mappers

```
// src/utils/mappers.ts
export function mapApiUser(apiUser: any) {
  return {
    id: apiUser.id,
    name: `${apiUser.first_name} ${apiUser.last_name}`,
    email: apiUser.email_address,
    role: apiUser.user_role?.toLowerCase() ?? 'user',
    isActive: apiUser.status === 'ACTIVE',
    createdAt: new Date(apiUser.created_at),
  }
}

export function groupBy<T>(arr: T[], key: keyof T): Record<string, T[]> {
  return arr.reduce((acc, item) => {
    const group = String(item[key])
    if (!acc[group]) acc[group] = []
    acc[group].push(item)
    return acc
  }, {} as Record<string, T[]>)
}
```

```
// src/utils/mappers.test.ts
import { mapApiUser, groupBy } from './mappers'

describe('mapApiUser', () => {
  const apiUser = {
    id: 1,
    first_name: 'John',
    last_name: 'Doe',
    email_address: 'john@example.com',
    user_role: 'ADMIN',
    status: 'ACTIVE',
    created_at: '2024-01-01T00:00:00Z',
  }

  test('maps all fields correctly', () => {
    const result = mapApiUser(apiUser)
    expect(result).toEqual({
      id: 1,
      name: 'John Doe',
      email: 'john@example.com',
      role: 'admin',
      isActive: true,
      createdAt: new Date('2024-01-01T00:00:00Z'),
    })
  })

  test('maps inactive status', () => {
    expect(mapApiUser({ ...apiUser, status: 'INACTIVE' }).isActive).toBe(false)
  })

  test('defaults role to user when null', () => {
    expect(mapApiUser({ ...apiUser, user_role: null }).role).toBe('user')
  })
})

describe('groupBy', () => {
  const items = [
    { id: 1, category: 'fruit', name: 'Apple' },
    { id: 2, category: 'veg', name: 'Carrot' },
    { id: 3, category: 'fruit', name: 'Banana' },
  ]
}
```

```

test('groups items by key', () => {
  const result = groupBy(items, 'category')
  expect(result.fruit).toHaveLength(2)
  expect(result.veg).toHaveLength(1)
  expect(result.fruit[0].name).toBe('Apple')
})

test('handles empty array', () => {
  expect(groupBy([], 'category')).toEqual({})
})
})

```

15.5 Auth Helpers

```

// src/utils/auth.ts
export function parseJwt(token: string): Record<string, any> | null {
  try {
    const base64 = token.split('.')[1].replace(/-/g, '+').replace(/_/g, '/')
    return JSON.parse(atob(base64))
  } catch {
    return null
  }
}

export function isTokenExpired(token: string): boolean {
  const payload = parseJwt(token)
  if (!payload?.exp) return true
  return Date.now() / 1000 > payload.exp
}

export function hasPermission(userRole: string, requiredRoles: string[]): boolean {
  return requiredRoles.includes(userRole)
}

```

```

// src/utils/auth.test.ts
import { parseJwt, isTokenExpired, hasPermission } from './auth'

// A real JWT payload (header.payload.signature)
const makeToken = (payload: object) => {
  const encoded = btoa(JSON.stringify(payload))
  return `header.${encoded}.signature`
}

describe('parseJwt', () => {
  test('parses valid token', () => {
    const token = makeToken({ sub: '123', name: 'John' })
    expect(parseJwt(token)).toMatchObject({ sub: '123', name: 'John' })
  })

  test('returns null for invalid token', () => {
    expect(parseJwt('not.a.token.at.all')).toBeNull()
    expect(parseJwt('')).toBeNull()
  })
})

describe('isTokenExpired', () => {
  test('returns true for expired token', () => {
    const token = makeToken({ exp: Math.floor(Date.now() / 1000) - 3600 }) // 1hr ago
    expect(isTokenExpired(token)).toBe(true)
  })
})

```

```

test('returns false for valid token', () => {
  const token = makeToken({ exp: Math.floor(Date.now() / 1000) + 3600 }) // 1hr
  ahead
  expect(isTokenExpired(token)).toBe(false)
})

test('returns true when no exp field', () => {
  const token = makeToken({ sub: '123' })
  expect(isTokenExpired(token)).toBe(true)
})

describe('hasPermission', () => {
  test('returns true when role matches', () => {
    expect(hasPermission('admin', ['admin', 'superadmin'])).toBe(true)
  })

  test('returns false when role not in list', () => {
    expect(hasPermission('user', ['admin'])).toBe(false)
  })
})

```

15.6 API Helpers

```

// src/utils/api.ts
export function buildQueryString(params: Record<string, any>): string {
  const qs = new URLSearchParams()
  Object.entries(params).forEach(([key, value]) => {
    if (value !== undefined && value !== null && value !== '') {
      qs.set(key, String(value))
    }
  })
  const result = qs.toString()
  return result ? `?${result}` : ''
}

export function parseErrorResponse(error: any): string {
  if (error?.response?.data?.message) return error.response.data.message
  if (error?.message) return error.message
  return 'Something went wrong. Please try again.'
}

export async function retry<T>(fn: () => Promise<T>, times = 3, delayMs = 1000):
Promise<T> {
  try {
    return await fn()
  } catch (err) {
    if (times <= 1) throw err
    await new Promise(res => setTimeout(res, delayMs))
    return retry(fn, times - 1, delayMs)
  }
}

// src/utils/api.test.ts
import { buildQueryString, parseErrorResponse, retry } from './api'

describe('buildQueryString', () => {
  test('builds string from params', () => {
    expect(buildQueryString({ page: 1, limit: 10 })).toBe('?page=1&limit=10')
  })
})

```

```

    test('skips null, undefined, empty string', () => {
      expect(buildQueryString({ q: '', page: null, sort: undefined, limit: 10
    })).toBe('?limit=10')
    })

    test('returns empty string for empty object', () => {
      expect(buildQueryString({})).toBe('')
    })
  })

  describe('parseErrorResponse', () => {
    test('extracts nested API message', () => {
      const err = { response: { data: { message: 'Not found' } } }
      expect(parseErrorResponse(err)).toBe('Not found')
    })

    test('falls back to error.message', () => {
      expect(parseErrorResponse(new Error('Network error'))).toBe('Network error')
    })

    test('falls back to default message', () => {
      expect(parseErrorResponse(null)).toBe('Something went wrong. Please try again.')
      expect(parseErrorResponse({})).toBe('Something went wrong. Please try again.')
    })
  })

  describe('retry', () => {
    jest.useFakeTimers()

    test('resolves on first try', async () => {
      const fn = jest.fn().mockResolvedValue('data')
      await expect(retry(fn, 3)).resolves.toBe('data')
      expect(fn).toHaveBeenCalledTimes(1)
    })

    test('retries on failure then resolves', async () => {
      const fn = jest.fn()
        .mockRejectedValueOnce(new Error('fail'))
        .mockResolvedValue('data')

      const promise = retry(fn, 3, 100)
      jest.runAllTimers()

      await expect(promise).resolves.toBe('data')
      expect(fn).toHaveBeenCalledTimes(2)
    })

    test('throws after all retries exhausted', async () => {
      const fn = jest.fn().mockRejectedValue(new Error('always fails'))

      const promise = retry(fn, 3, 100)
      jest.runAllTimers()

      await expect(promise).rejects.toThrow('always fails')
      expect(fn).toHaveBeenCalledTimes(3)
    })
  })
})

```

15.7 test.each — Table-Driven Tests

• • •

```
// test.each is perfect for utilities with many input/output combos
```

```

describe('formatCurrency - table driven', () => {
  test.each([
    [0, 'INR', '₹0.00'],
    [1000, 'INR', '₹1,000.00'],
    [100000, 'INR', '₹1,00,000.00'],
    [1000, 'USD', '$1,000.00'],
    [-500, 'INR', '-₹500.00'],
  ])('formatCurrency(%d, %s) → %s', (amount, currency, expected) => {
    expect(formatCurrency(amount, currency)).toBe(expected)
  })
})

// Object variant - gives named columns in output
describe('isValidEmail - table driven', () => {
  test.each([
    { email: 'valid@test.com', expected: true },
    { email: 'UPPER@TEST.COM', expected: true },
    { email: 'invalid', expected: false },
    { email: '', expected: false },
    { email: 'missing@', expected: false },
  ])('isValidEmail($email) → $expected', ({ email, expected }) => {
    expect(isValidEmail(email)).toBe(expected)
  })
})

```

Chapter 16 — Real Production App — What to Test

A production React app has many moving parts. This chapter maps every common feature to what tests to write and at what level.

16.1 Authentication Flow

```
// Tests to write for Auth:
// 1. Login form submits and stores token
// 2. Invalid credentials shows error
// 3. Token expiry redirects to login
// 4. Protected routes redirect unauthenticated users
// 5. Logout clears token and redirects

test('login stores token in localStorage', async () => {
  const user = userEvent.setup()
  render(<LoginPage />)

  await user.type(screen.getByLabelText(/email/i), 'john@test.com')
  await user.type(screen.getByLabelText(/password/i), 'Password@123')
  await user.click(screen.getByRole('button', { name: /login/i }))

  await waitFor(() => {
    expect(localStorage.getItem('token')).toBeTruthy()
  })
  // Should redirect to dashboard
  expect(screen.getByRole('heading', { name: /dashboard/i })).toBeInTheDocument()
})

test('shows error for wrong credentials', async () => {
  server.use(
    http.post('/api/auth/login', () =>
      HttpResponse.json({ message: 'Invalid email or password' }, { status: 401 })
    )
  )
  const user = userEvent.setup()
  render(<LoginPage />)

  await user.type(screen.getByLabelText(/email/i), 'wrong@test.com')
  await user.type(screen.getByLabelText(/password/i), 'wrongpass')
  await user.click(screen.getByRole('button', { name: /login/i }))

  expect(await screen.findByRole('alert')).toHaveTextContent('Invalid email or password')
  expect(localStorage.getItem('token')).toBeNull()
})

test('ProtectedRoute redirects unauthenticated user', () => {
  // No token in localStorage
  render(
    <MemoryRouter initialEntries={['/dashboard']}>
      <Routes>
        <Route path="/login" element={<LoginPage />} />
        <Route path="/dashboard" element={<ProtectedRoute><Dashboard /></ProtectedRoute>} />
      </Routes>
    </MemoryRouter>
  )
  expect(screen.getByRole('heading', { name: /login/i })).toBeInTheDocument()
})
```

```

test('logout clears token and redirects', async () => {
  localStorage.setItem('token', 'valid-token')
  const user = userEvent.setup()
  render(<Dashboard />)

  await user.click(screen.getByRole('button', { name: /logout/i }))

  expect(localStorage.getItem('token')).toBeNull()
  expect(screen.getByRole('heading', { name: /login/i })).toBeInTheDocument()
})

```

16.2 Data Table with Search, Sort, Pagination

```

// Tests to write:
// 1. Renders rows from API
// 2. Search filters rows
// 3. Sort changes order
// 4. Pagination loads next page
// 5. Empty state when no results
// 6. Loading state
// 7. Error state

test('renders data table rows', async () => {
  render(<UsersTable />)
  const rows = await screen.findAllByRole('row')
  expect(rows).toHaveLength(11) // 10 data rows + 1 header
})

test('search filters results', async () => {
  const user = userEvent.setup()
  render(<UsersTable />)
  await screen.findAllByRole('row') // wait for initial load

  await user.type(screen.getByRole('searchbox'), 'john')

  // Table re-fetches with search param
  await waitFor(() => {
    expect(screen.getAllByRole('row')).toHaveLength(2) // header + 1 result
  })
})

test('sort by name changes order', async () => {
  const user = userEvent.setup()
  render(<UsersTable />)
  await screen.findAllByRole('row')

  await user.click(screen.getByRole('columnheader', { name: /name/i }))

  const rows = await screen.findAllByRole('row')
  const firstCell = rows[1].querySelector('td')
  expect(firstCell).toHaveTextContent('Alice') // sorted alphabetically
})

test('shows empty state when no data', async () => {
  server.use(http.get('/api/users', () => HttpResponse.json({ data: [], total: 0 })))
  render(<UsersTable />)
  expect(await screen.findByText(/no users found/i)).toBeInTheDocument()
})

```

16.3 CRUD Operations

```

// Tests for a full CRUD feature (e.g. Product management):
// Create / Read / Update / Delete

describe('Product CRUD', () => {
  test('CREATE - form submits and new item appears in list', async () => {
    const user = userEvent.setup()
    render(<ProductsPage />)

    await user.click(screen.getByRole('button', { name: /add product/i }))
    await user.type(screen.getByLabelText(/product name/i), 'New Widget')
    await user.type(screen.getByLabelText(/price/i), '999')
    await user.click(screen.getByRole('button', { name: /save/i }))

    expect(await screen.findByText('New Widget')).toBeInTheDocument()
    expect(screen.queryByRole('dialog')).not.toBeInTheDocument()
  })

  test('UPDATE - edit form pre-fills and saves changes', async () => {
    const user = userEvent.setup()
    render(<ProductsPage />)
    await screen.findAllByRole('row')

    await user.click(screen.getAllByRole('button', { name: /edit/i })[0])

    // Pre-filled form
    expect(screen.getByLabelText(/product name/i)).toHaveValue('Existing Product')

    await user.clear(screen.getByLabelText(/product name/i))
    await user.type(screen.getByLabelText(/product name/i), 'Updated Name')
    await user.click(screen.getByRole('button', { name: /save/i }))

    expect(await screen.findByText('Updated Name')).toBeInTheDocument()
  })

  test('DELETE - confirm dialog then item removed', async () => {
    const user = userEvent.setup()
    render(<ProductsPage />)
    await screen.findAllByRole('row')

    await user.click(screen.getAllByRole('button', { name: /delete/i })[0])

    // Confirmation dialog appears
    expect(screen.getByRole('dialog')).toHaveTextContent(/are you sure/i)

    await user.click(screen.getByRole('button', { name: /confirm/i }))

    await waitForElementToBeRemoved(() => screen.queryByText('Product to Delete'))
  })

  test('DELETE - cancel does not remove item', async () => {
    const user = userEvent.setup()
    render(<ProductsPage />)
    await screen.findAllByRole('row')
    const initialRows = screen.getAllByRole('row').length

    await user.click(screen.getAllByRole('button', { name: /delete/i })[0])
    await user.click(screen.getByRole('button', { name: /cancel/i }))

    expect(screen.getAllByRole('row')).toHaveLength(initialRows) // unchanged
  })
})

```

16.4 File Upload Feature

```
test('file upload shows preview and sends file', async () => {
  const user = userEvent.setup()
  let receivedFormData

  server.use(
    http.post('/api/upload', async ({ request }) => {
      receivedFormData = await request.formData()
      return HttpResponse.json({ url: 'https://cdn.example.com/file.png' })
    })
  )

  render(<FileUploader />)

  const file = new File(['image content'], 'photo.png', { type: 'image/png' })
  await user.upload(screen.getByLabelText(/upload file/i), file)

  // Preview appears
  expect(screen.getByRole('img', { name: /preview/i })).toBeInTheDocument()

  await user.click(screen.getByRole('button', { name: /upload/i }))

  // Success message
  expect(await screen.findByText(/upload successful/i)).toBeInTheDocument()
  expect(receivedFormData.get('file')).toBeTruthy()
})

test('rejects files over size limit', async () => {
  const user = userEvent.setup()
  render(<FileUploader maxSizeMB={1} />)

  // Create 2MB file
  const bigFile = new File([new ArrayBuffer(2 * 1024 * 1024)], 'big.png', { type: 'image/png' })
  await user.upload(screen.getByLabelText(/upload file/i), bigFile)

  expect(screen.getByRole('alert')).toHaveTextContent(/file too large/i)
})
```

16.5 Notifications / Toast System

```
test('success toast appears after action', async () => {
  const user = userEvent.setup()
  render(<App />) // App includes ToastProvider

  await user.click(screen.getByRole('button', { name: /save settings/i }))

  const toast = await screen.findByRole('status') // role="status" for toasts
  expect(toast).toHaveTextContent('Settings saved successfully')

  // Auto-dismisses after timeout
  await waitForElementToBeRemoved(() => screen.queryByRole('status'), { timeout: 5000 })
})

test('error toast appears on API failure', async () => {
  server.use(http.post('/api/settings', () => new HttpResponse(null, { status: 500 })))
})
```

```

const user = userEvent.setup()
render(<SettingsPage />)

await user.click(screen.getByRole('button', { name: /save/i }))

expect(await screen.findByRole('alert')).toHaveTextContent(/failed to save/i)
})

```

16.6 Role-Based UI (RBAC)

```

// Users see different UI based on role

test('admin sees delete button, user does not', () => {
  render(<ProductsPage />, { user: { role: 'admin' } })
  expect(screen.getByRole('button', { name: /delete/i })).toBeInTheDocument()
})

test('regular user cannot see admin controls', () => {
  render(<ProductsPage />, { user: { role: 'user' } })
  expect(screen.queryByRole('button', { name: /delete/i })).not.toBeInTheDocument()
  expect(screen.queryByRole('button', { name: /add product/i })).not.toBeInTheDocument()
})

// Utility test for hasPermission function
describe('hasPermission', () => {
  test.each([
    ['admin', ['admin'], true],
    ['admin', ['admin', 'editor'], true],
    ['user', ['admin'], false],
    ['superadmin', ['admin'], false],
  ])('role %s, required %j → %s', (role, required, expected) => {
    expect(hasPermission(role, required)).toBe(expected)
  })
})

```

16.7 Theme / Dark Mode Toggle

```

test('toggles between light and dark theme', async () => {
  const user = userEvent.setup()
  render(<App />)

  const toggle = screen.getByRole('switch', { name: /dark mode/i })

  // Starts light
  expect(document.body).toHaveClass('theme-light')
  expect(toggle).not.toBeChecked()

  // Toggle to dark
  await user.click(toggle)
  expect(document.body).toHaveClass('theme-dark')
  expect(toggle).toBeChecked()
  expect(localStorage.getItem('theme')).toBe('dark')
})

test('persists theme preference on reload', () => {
  localStorage.setItem('theme', 'dark')
  render(<App />)
  expect(document.body).toHaveClass('theme-dark')
})

```

```
})
```

16.8 Language / i18n

```
import { I18nextProvider } from 'react-i18next'
import i18n from '../i18n'

function renderWithI18n(ui, { lng = 'en' } = {}) {
  i18n.changeLanguage(lng)
  return render(<I18nextProvider i18n={i18n}>{ui}</I18nextProvider>)
}

test('renders English labels', () => {
  renderWithI18n(<LoginPage />, { lng: 'en' })
  expect(screen.getByLabelText(/email/i)).toBeInTheDocument()
  expect(screen.getByRole('button', { name: 'Login' })).toBeInTheDocument()
})

test('renders translated labels', () => {
  renderWithI18n(<LoginPage />, { lng: 'ta' }) // Tamil
  expect(screen.getByRole('button', { name: 'தமிழ்' })).toBeInTheDocument()
})
```

Chapter 17 — Code Coverage — Complete Guide

17.1 What Coverage Measures

Metric	What it counts
Statements	Each executable statement — was it run?
Branches	Each if/else/ternary/switch arm — was each branch taken?
Functions	Each function/method — was it called?
Lines	Each line — was it executed?

NOTE

Branch coverage is the most valuable. A function can have 100% line coverage but 0% branch coverage if all your tests always take the same if path.

17.2 Running Coverage

```
# Jest
npx jest --coverage
npx jest --coverage --coverageReporters=text      # terminal only
npx jest --coverage --coverageReporters=html     # open coverage/index.html

# Vitest
npx vitest run --coverage
npx vitest --ui      # see coverage inline in browser UI

# CI - enforce thresholds, fail if below
npx jest --ci --coverage # fails if thresholds not met
```

17.3 Coverage Configuration

```
// jest.config.js - complete coverage setup
module.exports = {
  collectCoverageFrom: [
    'src/**/*..{js,jsx,ts,tsx}',
    '!src/**/*.d.ts',           // skip type files
    '!src/index.tsx',           // skip entry point
    '!src/main.tsx',
    '!src/**/*.stories.*',     // skip Storybook
    '!src/test/**',             // skip test helpers
    '!src/mocks/**',            // skip MSW mocks
    '!src/**/*.__fixtures__/**', // skip fixtures
  ],
  coverageThreshold: {
    // Global minimum - fails CI if below
    global: {
      branches: 70,
      functions: 80,
      lines: 80,
      statements: 80,
    },
    // Per-file minimum (optional - strict mode)
    './src/utils/': {
      branches: 90, // utilities must be well-tested
      functions: 90,
    },
  },
}
```

```

    lines: 90,
  },
},

coverageReporters: ['text', 'text-summary', 'html', 'lcov'],
// lcov → uploads to Codecov, SonarQube, etc.
}

```

17.4 Reading the Coverage Report

Terminal output example:

File	% Stmts	% Branch	% Funcs	% Lines	
All files	82.5	71.4	86.7	83.1	
components/					
Counter.tsx	100	100	100	100	
LoginForm.tsx	90.9	75.0	85.7	90.0	← branches not covered
utils/					
format.ts	100	100	100	100	
validators.ts	66.7	50.0	75.0	66.7	← needs more tests
api.ts	45.0	30.0	50.0	45.0	← heavily undertested

❏ TIP

Open `coverage/index.html` in browser to see exactly which lines are red (uncovered). Click into any file — red = not run, yellow = partial branch, green = covered.

17.5 What Coverage Does NOT Tell You

- High coverage does not mean tests are good — you can cover code with useless assertions
- A test that renders a component and asserts nothing gives coverage with zero confidence
- Coverage does not check edge cases — only that the code ran
- Aim for meaningful coverage: every branch tested with real assertions

❏ WARN

Never write tests just to increase the coverage number. A test with no `expect()` is worse than no test — it gives false confidence.

17.6 Excluding From Coverage

```

// Exclude specific lines with inline comments
/* istanbul ignore next */    // Jest (Istanbul)
/* v8 ignore next */         // Vitest (v8)

// Example — skip an unreachable default branch
function getLabel(status: string) {
  if (status === 'active') return 'Active'
  if (status === 'inactive') return 'Inactive'
  /* istanbul ignore next */
  return 'Unknown'    // defensive fallback — hard to test
}

// Exclude entire file via config

```

```
coveragePathIgnorePatterns: [  
  '/node_modules/',  
  'src/generated/', // auto-generated API types  
  'src/mocks/',  
]
```

Chapter 18 — CI/CD — Tests Before Deployment

Tests only protect you if they run automatically on every code change. This chapter shows how to wire tests into your pipeline so nothing broken ever ships.

18.1 The Testing Pipeline

- 1. Pre-commit — lint + type check (fast, blocks bad commits locally)
- 2. PR/Push — full test suite + coverage (blocks merging broken code)
- 3. Pre-deploy — E2E smoke tests (blocks broken deploys to production)
- 4. Post-deploy — synthetic monitoring (catches production-only issues)

18.2 GitHub Actions — Full CI Pipeline

```
# .github/workflows/ci.yml
name: CI

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Setup Node
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - name: Install dependencies
        run: npm ci          # ci is faster + reproducible (uses lockfile
                             exactly)

      - name: Type check
        run: npx tsc --noEmit  # catch type errors before tests

      - name: Lint
        run: npm run lint

      - name: Run tests
        run: npm run test:ci    # jest --ci --coverage --watchAll=false

      - name: Upload coverage
        uses: codecov/codecov-action@v4
        with:
          token: ${ secrets.CODECOV_TOKEN }
          files: ./coverage/lcov.info
          fail_ci_if_error: true  # fail if coverage upload fails
```

18.3 package.json — All Test Scripts

```
{
  "scripts": {
    // Development
    "test": "jest --watch",
    "test:run": "jest --watchAll=false",

    // CI — no interactivity, exit code non-zero on failure
    "test:ci": "jest --ci --coverage --watchAll=false --forceExit",

    // Coverage only
    "test:coverage": "jest --coverage --watchAll=false",

    // Specific file
    "test:file": "jest --watchAll=false --testPathPattern",

    // Vitest equivalents
    "test": "vitest",
    "test:run": "vitest run",
    "test:ci": "vitest run --coverage --reporter=verbose",
    "test:ui": "vitest --ui"
  }
}
```

18.4 Pre-commit Hook with Husky

```
# Install
npm install --save-dev husky lint-staged
npx husky init

# .husky/pre-commit
#!/usr/bin/env sh
. "$(dirname -- "$0")/_/husky.sh"
npx lint-staged
```

```
// package.json — lint-staged config
{
  "lint-staged": {
    "*.ts,tsx,js,jsx": [
      "eslint --fix",
      "prettier --write"
    ],
    "*.ts,tsx": [
      "bash -c 'tsc --noEmit'"
    ]
  }
}
```

NOTE

Pre-commit hooks run related tests only (lint-staged). Full test suite runs in CI. Never block commits with full test suite — it kills developer flow.

18.5 Running Tests on Code Change — Checklist

When	What to Run
Before committing locally	npx jest --watch (related files only, auto-mode)

Before pushing a PR	<code>npx jest --ci</code> (all tests, no watch)
After adding a new component	<code>npx jest ComponentName.test</code> (just that file)
After changing a utility	<code>npx jest utils/</code> (all util tests)
After refactoring	<code>npx jest --coverage</code> (verify nothing broke, check branches)
Before deploying	<code>npm run test:ci</code> (full suite + coverage thresholds)
After deploy	Cypress/Playwright smoke test against live environment

18.6 How to Verify Tests After Changing Code

```

# Scenario: You changed src/utils/format.ts

# 1. Run only related tests - instant feedback
npx jest format

# 2. Run with coverage to see if you missed branches
npx jest format --coverage

# 3. Check for regressions across entire codebase
npx jest --ci

# 4. Run in watch mode while working
npx jest --watch # press 'p' → filter by filename, 't' → filter by test name

# Vitest equivalents
npx vitest run src/utils/format
npx vitest related src/utils/format.ts # only tests related to this file
npx vitest --ui # visual filter in browser

```

18.7 Debugging Failing Tests in CI

```

# 1. Reproduce locally with same flags as CI
npm run test:ci

# 2. Run with verbose output
npx jest --verbose --no-coverage

# 3. Run failing test in isolation
npx jest -t "formats INR correctly" # by test name
npx jest src/utils/format.test.ts # by file

# 4. Add screen.debug() to see what DOM looks like when test fails
test('renders button', () => {
  render(<MyComponent />)
  screen.debug() // prints DOM - commit this to see in CI logs if needed
  expect(screen.getByRole('button')).toBeInTheDocument()
})

# 5. Check for environment differences
# CI uses: NODE_ENV=test TERM=dumb CI=true
# Add console.log(process.env.NODE_ENV) if suspect env issue

```

18.8 Test Performance — Keep CI Fast

Technique	Impact
Run tests in parallel (default in Jest/Vitest)	3-5x faster on multi-core CI
Use --testPathPattern to run only changed files	Near instant feedback in PR
Separate unit and E2E into different CI jobs	Unit runs in 30s, E2E in parallel
Cache node_modules in CI	Save 60-90s on every run
Use Vitest instead of Jest for Vite projects	2-5x faster cold start
Mock heavy dependencies (DB, S3, email)	Prevents slow/flaky tests
Set test timeout low (5s default is too high)	Catches hanging tests fast
Use --bail on CI to stop after first failure	Saves time when build is broken

Chapter 19 — Best Practices & Golden Rules

19.1 The Testing Trophy

Kent C. Dodds prefers the Testing Trophy over the Testing Pyramid for modern React apps:

- Static Analysis (TypeScript, ESLint) — catches errors before tests even run
- Unit Tests — utilities, pure functions, isolated logic
- Integration Tests — RTL component tests with MSW — the biggest investment
- E2E Tests — Cypress/Playwright — a few critical user journeys only

INFO

Spend 70% of your testing effort on integration tests (RTL + MSW). They give the best confidence-to-cost ratio.

19.2 What to Test — Decision Framework

Code Type	Test Level	Key Focus
Utility functions	Unit — Jest/Vitest	All branches, edge cases, invalid input
Custom hooks	Unit — renderHook()	Initial state, state changes, cleanup
Presentational components	Integration — RTL	Renders with props, conditional rendering
Feature components	Integration — RTL + MSW	Full user flow, loading, error, success
Context providers	Integration — RTL	Value provided, updates propagate
Redux slices/RTK Query	Unit + Integration	Actions, reducers, selectors, API calls
API helper functions	Unit	buildQueryString, mappers, parsers
Critical user journeys	E2E — Cypress	Login, checkout, core workflows
Auth flow	Integration + E2E	Token storage, redirect, RBAC

19.3 What NOT to Test

- Third-party library internals — test your usage, not the library
- React itself — don't test that useState works
- Implementation details — internal state names, private methods, DOM structure
- Exact CSS pixel values — use visual regression tools instead
- Auto-generated code — GraphQL types, Prisma models
- Type definitions — TypeScript catches these

19.4 Naming Conventions

```
// Test files co-located with source (Vitest style)
src/
```

```

components/
  Button.tsx
  Button.test.tsx    // same folder
utils/
  format.ts
  format.test.ts

// OR in __tests__ folder (Jest convention)
src/
  __tests__/
    components/
      Button.test.tsx
    utils/
      format.test.ts

// Test naming - describe what the USER experiences
// ✗ BAD - technical
test('onClick prop fires', ...)
test('state changes to true', ...)

// ✓ GOOD - user-centric
test('clicking submit button calls onSubmit handler', ...)
test('checkbox becomes checked after clicking it', ...)
test('shows error message when email is invalid', ...)

```

19.5 Arrange-Act-Assert Pattern

```

test('submitting login form calls onSubmit with credentials', async () => {
  // — ARRANGE — set up component and data —————
  const user = userEvent.setup()
  const onSubmit = jest.fn()
  const { username, password } = buildLoginForm()
  render(<LoginForm onSubmit={onSubmit} />)

  // — ACT — perform user actions —————
  await user.type(screen.getByLabelText(/email/i), username)
  await user.type(screen.getByLabelText(/password/i), password)
  await user.click(screen.getByRole('button', { name: /login/i }))

  // — ASSERT — verify outcomes —————
  expect(onSubmit).toHaveBeenCalledTimes(1)
  expect(onSubmit).toHaveBeenCalledWith({ username, password })
})

```

19.6 One Test, One Behaviour

```

// ✗ BAD - tests too many things, hard to diagnose failure
test('form works', async () => {
  render(<LoginForm />)
  // ... fills form
  expect(submitBtn).toBeDisabled() // testing initial state
  // ... types email
  expect(emailInput).toHaveValue('test') // testing intermediate state
  // ... submits
  expect(fn).toHaveBeenCalledTimes(1) // testing submission
  expect(screen.getByText('Welcome')).toBeInTheDocument() // testing success
})

// ✓ GOOD - each test covers one behaviour
test('submit button is disabled when form is empty', ...)
test('submit button enables when all fields filled', ...)

```

```
test('submitting calls onSubmit with form values', ...)
test('shows welcome message after successful login', ...)
```

19.7 Keep Tests Independent

```

● ● ●
// ✗ BAD — tests depend on each other (order matters)
let result
test('first test creates data', () => { result = createUser() })
test('second test uses result', () => { expect(result.id).toBe(1) })

// ✔ GOOD — each test sets up its own data
test('user has id', () => {
  const result = createUser()
  expect(result.id).toBe(1)
})

// Shared setup belongs in beforeEach
let user
beforeEach(() => {
  user = userEvent.setup() // fresh instance per test
})

```

19.8 Mock at the Right Level

Situation	What to Mock
Testing a component that fetches data	MSW — intercept HTTP (not fetch/axios directly)
Testing a component that uses a hook	Don't mock hooks — test through them
Testing a hook that calls an API	MSW — intercept HTTP in the hook test
Testing a utility that uses Date.now()	jest.spyOn(Date, "now").mockReturnValue(...)
Testing browser APIs (geolocation, etc.)	Mock window.navigator.geolocation directly
Testing third-party hook (useGeolocation)	jest.mock("react-use-geolocation")
Testing file system / DB	Always mock — use MSW or test doubles

19.9 The Pre-Deploy Test Checklist

```

● ● ●
# Before every deployment — run this sequence:

# 1. Type check (catches compile errors)
npx tsc --noEmit

# 2. Lint (catches code quality issues)
npm run lint

# 3. Full test suite with coverage
npm run test:ci

# 4. Build succeeds
npm run build

```

```
# 5. E2E smoke test against staging
npx cypress run --env baseUrl=https://staging.yourapp.com

# If all pass → safe to deploy
# If any fail → DO NOT deploy — fix first
```

19.10 The Code Change Verification Checklist

Change Type	Tests to Run
Changed a utility function	Unit tests for that file + any component using it
Changed a component	That component's test + any parent that renders it
Changed an API endpoint / route	MSW handlers + all tests that call that endpoint
Changed CSS / Styling	Visual regression test or manual check (not unit tests)
Changed auth logic	Auth tests + ProtectedRoute tests + E2E login flow
Added a new feature	New tests for new feature + regression suite
Refactored without behaviour change	Full suite — confirms behaviour unchanged
Upgraded a dependency	Full suite + check for deprecation warnings
Changed environment config	Test in CI — local env can hide issues

19.11 Common Anti-Patterns to Avoid

```
// ❌ 1. Testing implementation details
expect(component.state.isLoading).toBe(false) // access internal state

// ❌ 2. Querying by DOM structure
container.querySelector('.btn-primary') // CSS class coupling

// ❌ 3. Ignoring act() warnings
// "not wrapped in act" → missing await, fix don't suppress

// ❌ 4. Giant describe blocks with 50 tests
// Split by feature or behaviour

// ❌ 5. expect() with no assertion
test('renders', () => {
  render(<MyComponent />)
  // no expect — gives coverage but zero confidence
})

// ❌ 6. Mocking too much
jest.mock('../api')
jest.mock('../utils')
jest.mock('../hooks') // you're now testing mocks, not your code

// ❌ 7. waitFor wrapping userEvent
await waitFor(() => userEvent.click(button)) // wrong — waitFor is for
assertions

// ✅ Correct
await userEvent.click(button)
await waitFor(() => expect(screen.getByText('done')).toBeInTheDocument())
```

